



Degree Project in Computer Science and Engineering

Second cycle, 30 credits

Optimistic Consensus under Conformal Risk Control

PAUL MICHAEL KILIAN

Optimistic Consensus under Conformal Risk Control

PAUL MICHAEL KILIAN

Master's Programme, Software Engineering of Distributed Systems, 120 credits
Date: September 2, 2026

Supervisors: Sonia Horchidan, Harald Ng
Examiner: Paris Carbone

Swedish title: Optimistisk konsensus med konform riskkontroll

Abstract

State Machine Replication (SMR) is a core technique in distributed systems, forming the basis for critical services like strongly consistent distributed databases. It fundamentally builds upon totally ordered, replicated logs provided by *consensus* algorithms. While fast-path consensus algorithms like *Fast Paxos* eliminate a communication delay under favorable conditions, they are inherently optimistic. When multiple proposals are submitted concurrently, collisions occur, triggering expensive recovery mechanisms that can degrade end-to-end performance below that of conservative protocols.

This thesis introduces *Conformal Consensus* as an exploratory hybrid protocol that bridges distributed consensus with statistical uncertainty quantification. The protocol allows individual nodes to dynamically and independently choose between a conservative and a fast execution path by evaluating locally observable network metrics, such as latencies and proposal rates, whereas a conformal wrapper calibrates decision thresholds such that the long-term empirical rate of fast-path errors remains mathematically bounded to a user-specified target.

Experimental results from controlled distributed deployments show that the conformal mechanism successfully tracks the target error rate with negligible compute overhead. While small improvements in end-to-end latency can be observed under favorable, low-contention workloads, the practicality of such a hybrid protocol for production deployment is limited due to inherent fast-path viability problems for replicated logs. Ultimately, this work provides an exploration into applying statistical risk control to distributed protocols, highlighting that conformal methods can successfully govern protocol-level trade-offs in volatile network environments.

Keywords

State Machine Replication, Consensus, Paxos, Fast Paxos, Conformal Risk Control

Sammanfattning

Tillståndsmaskinsreplikering (SMR) är en central teknik inom distribuerade system och utgör grunden för kritiska tjänster såsom starkt konsistenta distribuerade databaser. Den bygger i grunden på totalt ordnade, replikerade loggar som tillhandahålls av konsensusalgoritmer. Även om konsensusalgoritmer med snabbspår, såsom *Fast Paxos*, eliminerar ett kommunikationssteg under gynnsamma förhållanden, är de till sin natur optimistiska. När flera proposals skickas in samtidigt uppstår kollisioner, vilket utlöser kostsamma återhämtningsmekanismer som kan försämra end-to-end-latensen till en nivå som är sämre än för mer konservativa protokoll.

Denna avhandling introducerar *Conformal Consensus* som ett utforskande hybridprotokoll som överbryggar distribuerad konsensus och statistisk osäkerhetskvantifiering. Protokollet gör det möjligt för enskilda noder att dynamiskt och självständigt välja mellan en konservativ och en snabb exekveringsväg genom att utvärdera lokalt observerbara nätverksmått, såsom latenser och proposal-frekvenser. Samtidigt kalibrerar en konform wrapper beslutsgränserna så att den långsiktiga empiriska andelen fel i snabbspåret förblir matematiskt begränsad till ett användarspecifierat mål.

Experimentella resultat från kontrollerade distribuerade miljöer visar att den konforma mekanismen framgångsrikt följer den specificerade felfrekvensen med försumbar beräkningsmässig overhead. Även om små förbättringar i end-to-end-latens kan observeras under gynnsamma arbetsbelastningar med låg konfliktnivå, begränsas den praktiska användbarheten av ett sådant hybridprotokoll för produktionsanvändning av inneboende problem med snabbspårets praktiska användbarhet för replikerade loggar. Sammantaget utgör detta arbete en utforskning av hur statistisk riskkontroll kan tillämpas på distribuerade protokoll och visar att konforma metoder framgångsrikt kan styra avvägningar på protokollnivå i föränderliga nätverksmiljöer.

Nyckelord

Tillståndsmaskinsreplikering, Konsensus, Paxos, Fast Paxos, Konform riskkontroll

Zusammenfassung

Zustandsmaschinenreplikation (SMR) ist eine zentrale Technik in verteilten Systemen und bildet die Grundlage für kritische Dienste wie stark konsistente verteilte Datenbanken. Sie basiert grundlegend auf total geordneten, replizierten Logs, die durch Konsensalgorithmen bereitgestellt werden. Obwohl schnelle Ausführungspfade in Konsensalgorithmen wie *Fast Paxos* unter günstigen Bedingungen einen Kommunikationsschritt einsparen können, sind sie von Natur aus optimistisch. Wenn mehrere Proposals gleichzeitig eingebracht werden, treten Kollisionen auf, die teure Wiederherstellungsmechanismen auslösen können. Dadurch kann die Ende-zu-Ende-Latenz sogar schlechter ausfallen als bei konservativeren Protokollen.

Diese Arbeit führt *Conformal Consensus* als ein exploratives Hybridprotokoll ein, das verteilten Konsens mit statistischer Unsicherheitsquantifizierung verbindet. Das Protokoll ermöglicht es einzelnen Instanzen, dynamisch und unabhängig zwischen einem konservativen und einem schnellen Ausführungspfad zu wählen, indem lokal beobachtbare Netzwerkmetriken wie Latenzen und Proposal-Raten ausgewertet werden. Gleichzeitig kalibriert ein konformer Wrapper die Entscheidungsschwellen so, dass die langfristige empirische Fehlerrate des schnellen Pfads mathematisch auf ein vom Benutzer vorgegebenes Ziel begrenzt bleibt.

Experimentelle Ergebnisse aus kontrollierten verteilten Umgebungen zeigen, dass der konforme Mechanismus die Ziel-Fehlerrate erfolgreich nachverfolgt und dabei nur vernachlässigbaren Rechenaufwand verursacht. Obwohl unter günstigen Arbeitslasten mit geringer Konkurrenz kleine Verbesserungen der Ende-zu-Ende-Latenz beobachtet werden können, ist die praktische Eignung eines solchen Hybridprotokolls für den produktiven Einsatz aufgrund inhärenter Probleme hinsichtlich der Nutzbarkeit schneller Pfade für replizierte Logs eingeschränkt. Insgesamt untersucht diese Arbeit die Anwendung statistischer Risikokontrolle auf verteilte Protokolle und zeigt, dass konforme Methoden erfolgreich dazu eingesetzt werden können, protokollbezogene dynamische Entscheidungen in volatilen Netzwerkumgebungen zu steuern.

Schlüsselwörter

Zustandsmaschinenreplikation, Konsens, Paxos, Fast Paxos, Konforme Risikokontrolle

Acknowledgments

I would like to express my gratitude to my supervisors, Sonia Horchidan and Harald Ng, for their guidance, support, and the sheer endless enthusiasm they showed throughout this process. Their contributions to this project cannot be overstated. I would also like to thank Paris Carbone for serving as the examiner of this thesis and for reigniting my interest in distributed systems.

Stockholm, October 2026

Paul Michael Kilian

Contents

1	Introduction	1
1.1	Research Questions	2
1.2	Challenges	3
1.3	Research Methodology	3
1.4	Delimitations	4
1.5	Thesis Outline	4
2	Background	5
2.1	State Machine Replication	5
2.1.1	Uniform Sequence Consensus	6
2.1.2	Leader-Based Sequence Paxos	8
2.1.3	Fast-Path Integration	10
2.2	Conformal Prediction	12
2.2.1	Full Conformal Prediction	13
2.2.2	Split Conformal Prediction	15
2.2.3	Conformal Risk Control	16
2.2.4	Adaptive Conformal Inference	18
2.3	Related Work	20
2.3.1	Modern Fast-Path Consensus Protocols	21
2.3.2	Adaptive Consensus	21
2.3.3	Conformal Methods for Data Systems	22
2.4	Summary	23
3	Conformal Consensus	25
3.1	Decentralized Mode Switching	25
3.1.1	Mode Interoperability	26
3.1.2	Quorum Logic and Collision Detection	27
3.2	Heuristic Score for Fast Path Success	31
3.2.1	Important Metrics	32

3.2.2	Score Function	34
3.3	Risk-Controlled Mode Prediction	37
3.3.1	Risk Specification	37
3.3.2	Decision Layer	39
3.3.3	Calibration Strategy	40
4	Analysis and Results	43
4.1	Experimental Setup	44
4.1.1	State Machine Application	45
4.1.2	Node Setup	46
4.1.3	Workload Pattern	48
4.2	Score Function Efficacy	51
4.3	Empirical Validation of Risk Control	52
4.4	Latency Study	53
4.5	Overhead Analysis	56
5	Discussion	59
5.1	The Power of Formal Guarantees	59
5.2	Fast Path Viability	60
5.3	Practicality of Conformal Consensus	61
5.4	Limitations	61
5.5	Future Work	62
6	Conclusions	63
	References	65
A	Proof of Slow-Path Resolution Safety	73
B	Declaration of AI Use	75

List of Figures

2.1	Architecture of a State Machine Replication (SMR) service using a replicated log abstraction.	6
2.2	Example execution of Paxos with three servers.	9
2.3	Example execution of leader-based Sequence Paxos with five servers. Chosen values can be broadcast to learners similar to before.	10
2.4	Example execution of leader-based Sequence Paxos with five servers using the fast path successfully.	11
2.5	Example execution of leader-based Sequence Paxos with five servers and a collision of proposals.	12
2.6	A depiction of split conformal prediction with a calibration set. The score of a new test feature has an equal probability to fall in between any two calibration scores when the correct label Y_{test} is plugged in. This actual score will be greater than the consistency threshold and therefore inconsistent with the calibration data with a probability of α	16
2.7	Empirical risk $\hat{R}(\lambda)$ as a function of the tuning parameter λ adapted from [29]. The selected value $\hat{\lambda}$ is the smallest parameter for which the empirical risk falls below the target risk level.	17
3.1	Feature importance for fast-path success measured by total information gain across the XGBoost model ensemble.	33
3.2	Feature importance for fast-path success with a reduced feature set measured by total information gain.	35
3.3	Observed fast-path success rate versus the expected number of competing proposals. The red line highlights the analytical heuristic score used for prediction.	36
4.1	European cluster setup, including one-way latencies.	45

4.2	US cluster setup, including one-way latencies.	46
4.3	Sample of the client workload pattern used for experimentation.	49
4.4	Heuristic score path selection of different nodes depending on their own load and other servers' load under a static threshold of 0.1.	50
4.5	Heuristic score path selection of different nodes depending on their own load and other servers' load under a static threshold of 0.5.	51
4.6	Comparison of the target fast-path error rate and the actual observed fast-path error rate across nodes in the US and European clusters.	53
4.7	End-to-end latency comparison for the European cluster setup between Regular Paxos, Fast Paxos, and Conformal Consensus with various risk levels.	54
4.8	End-to-end latency comparison for the (simulated) US cluster setup between Regular Paxos, Fast Paxos, and Conformal Consensus with various risk levels.	55
4.9	Average execution times of an append attempt by followers across operating modes.	56
4.10	Threshold computation time during batch calibration relative to calibration set size.	57

List of Tables

3.1	Performance comparison of feature selection and predictive models.	37
3.2	Loss based on true labels versus prediction sets.	38
4.1	Hardware and software configuration specifications for the experimental cluster instances deployed on GCP.	47
4.2	Estimated end-to-end latencies in the European cluster setup using regular, fast, or slow execution paths for an entry appended by a co-located client.	48
4.3	Estimated end-to-end latencies in the US cluster setup using regular, fast, or slow execution paths for an entry appended by a co-located client.	48

List of Algorithms

- 2.1 Full Conformal Prediction for Classification 14
- 2.2 Split Conformal Prediction for Classification 15
- 2.3 Conformal Risk Control for Classification 19
- 2.4 Rolling Risk Control for Classification 20
- 3.1 Quorum Evaluation and Collision Resolution 28
- 3.2 Leader Proposal Processing 30
- 3.3 Conformal Mode Prediction 40

List of Acronyms and Abbreviations

ACI	Adaptive Conformal Inference
BLE	Ballot Leader Election
CP	Conformal Prediction
CRC	Conformal Risk Control
FNR	False Negative Rate
GCP	Google Cloud Platform
ML	Machine Learning
RMSE	Root Mean Squared Error
RPS	Requests per Second
SMR	State Machine Replication
UQ	Uncertainty Quantification

Chapter 1

Introduction

Modern applications running on the Internet are increasingly distributed to enable scalability, improve service availability, and decrease user latency. In a distributed system, individual components can only coordinate via message passing over a network, making it naturally volatile and prone to partial failures [1]. This failure model and the asynchronous nature of communication networks, preventing a global notion of time, make many conceptually simple problems notoriously difficult. One is referred to as the *consensus* problem [1, 2, 3] and requires multiple processes to decide on the same value.

Research on this problem has come a long way since the original *Paxos* algorithm for solving consensus [4] was designed. *Fast Paxos*, for instance, introduced the *fast-path*, eliminating a one-way delay under favorable conditions [5, 6]. In practice, these algorithms are extended to maintain a continuously growing total-order *distributed log*, which enables the implementation of **State Machine Replication (SMR)** for processing deterministic operations on multiple nodes with guaranteed convergence to the same state [7]. This is an important abstraction for applications like distributed databases [8, 9, 10] and coordination services [11, 12].

While consensus-based distributed logs inherently provide strong safety guarantees, they can dominate end-to-end system performance regarding throughput and latency through their message complexity, particularly in geo-replicated deployments [13, 14]. As a result, a significant portion of current research focuses on reducing the overhead of log replication by using versions of the optimistic fast paths originally devised for Fast Paxos [15, 16, 17].

Fast-path algorithms are, however, inherently optimistic. They assume a lack of concurrent client proposals to bypass a communication step. This optimistic assumption fails in the event of a collision, which occurs when

multiple clients simultaneously propose conflicting values for the same log index. Resolving these collisions requires expensive, multi-round recovery mechanisms to maintain safety invariants. Under heavy contention, this recovery causes performance to degrade severely, rendering the system slower than a baseline, conservative protocol [18]. To circumvent these collision penalties, much of the current research on **SMR** departs from a single, totally ordered replicated log in favor of partially ordered or command-ordering mechanisms [14, 15, 16]. While this approach minimizes recovery costs, it forces the consensus layer to evaluate possibly complex application-level semantics to identify commuting commands. Furthermore, many applications may simply require strict total ordering.

An alternative, underexplored approach is to retain the totally ordered log as the core abstraction while regulating access to the fast path based on current conditions [18]. A system could gate fast-path attempts using a heuristic or predictive model, falling back to a conservative execution path when the risk of a conflict is high. Such a regulator must make decisions under uncertainty: allowing too many collisions increases overhead, while being overly conservative sacrifices potential latency improvements. *Conformal methods* [19, 20, 21] have emerged as a statistical framework for augmenting arbitrary predictive mechanisms with strict mathematical guarantees. While operating without assumptions about the underlying data distribution, conformal approaches calibrate decision thresholds based on observed outcomes and can regulate decision risk.

This thesis explores the usability of a hybrid consensus protocol that integrates decentralized path selection with conformal methods. In this *Conformal Consensus* design, individual followers independently evaluate local network conditions to dynamically decide between using an optimistic fast path or the conservative route, while a conformal wrapper attempts to control fast-path error rates.

1.1 Research Questions

Given the goal of bridging statistical risk control with distributed consensus, this work is guided by two research questions:

- I. *How can conformal methods be applied to regulate fast-path attempts in a replicated log abstraction while tracking a user-defined target for the error rate?*

- II. *To what extent can this approach reduce end-to-end latency compared to traditional, non-adaptive consensus baselines within distributed environments?*

1.2 Challenges

To answer these questions, the work must overcome several challenges, which define the primary objectives and contributions of this study:

1. Exploring the design of a "hybrid" consensus protocol for replicated logs where different nodes can independently and dynamically switch between fast and conservative operating modes.
2. Identifying which locally observable metrics have predictive capabilities of fast-path success and designing a practical heuristic.
3. Employing an online calibration method that can track a target rate of fast path mistakes.
4. Demonstrating through geo-replicated deployment if the adaptive methodology can improve end-to-end latency compared to baselines.
5. Determining the impact of risk regulation on the computational cost to prevent the regulator from becoming a performance bottleneck.

The application of conformal risk control to the domain of distributed consensus is, to our knowledge, novel. The findings could be of interest to distributed systems researchers who deal with the volatility of optimistic protocols in geo-replicated environments.

1.3 Research Methodology

We outline the research methodology following the structure suggested by Håkansson [22]. Accordingly, this thesis uses a quantitative research methodology, a positivist research paradigm, and a deductive research approach. This is appropriate because the research aims to answer research questions about the effect of fast-path regulation on system performance using objectively measurable variables such as latency and fast-path error rate. The study assumes that system behavior can be observed, measured, and analyzed statistically and that cause–effect relationships between protocol behavior and performance can be established through controlled experiments.

The primary research methods adopted in this thesis are experimental and empirical. The study is experimental because it involves implementing a regulator, manipulating system conditions, and measuring the resulting effects on performance under controlled conditions. It is empirical because the conclusions are based on observed data collected from simulations and experiments. To ensure research quality, the work addresses replicability through controlled experimental setups and publicly available artifacts. *

1.4 Delimitations

This thesis is highly exploratory, as there is little established methodology for designing and evaluating hybrid consensus approaches. Consequently, this work does not attempt to develop a fully fledged protocol with formal safety and liveness proofs. Instead, it focuses on observing system behavior under ideal conditions to determine whether a hybrid consensus approach is viable and has practical utility.

Regarding the system environment, this research assumes a fail-stop failure model without recovery from stable storage, thereby excluding the I/O overhead associated with durable logging. It does not address Byzantine faults, where malicious actors could intentionally circumvent the safety invariants of the consensus algorithms. Furthermore, the scope is restricted to software-level regulation at the consensus layer. It explicitly excludes hardware-based network optimizations or modifications to the underlying transport protocols.

Finally, this study is designed to isolate the overhead of the consensus mechanism. It does not account for complex application-level state machines, which could shift the overall performance bottleneck away from the network and the consensus protocol itself.

1.5 Thesis Outline

The remainder of this work is structured as follows: Chapter 2 establishes the theoretical foundations and related work. Chapter 3 presents the design aspects related to the hybrid protocol, the heuristic score, and the risk-controlled mode prediction. Chapter 4 shows the evaluation methodology and presents experimental results. The results are discussed and interpreted in Chapter 5 before Chapter 6 concludes the thesis.

*The library artifacts and reproducible experiment scripts are openly available under: <https://github.com/kilianpl4/conformal-consensus>

Chapter 2

Background

This chapter establishes the theoretical basis necessary to understand the implementation and analysis parts of this thesis. Section 2.1 introduces the concept of replicated state machines with their underlying consensus mechanisms. A specific focus lies on the difference between fast-path algorithms and conservative approaches for maintaining a replicated log. Section 2.2 presents an overview of conformal methods as a statistical framework for quantifying uncertainty before Section 2.3 reviews related work. Finally, Section 2.4 summarizes the discussed concepts.

2.1 State Machine Replication

A common way to provide fault tolerance and thereby improve availability for a service is to operate multiple servers that maintain an identical state. These replicas can serve as backups, shadowing a primary node and being ready to take over during a failure, or be potentially globally distributed to handle requests from local clients. Many services, like databases, require a strict notion of consistency and need nodes to coordinate their actions to prevent state divergence in both of these cases.

One approach is to decide on the execution of the same sequence of deterministic operations across all nodes. This is referred to as **SMR** [7]. The Raft protocol [23] popularized the concept of a replicated log as the central abstraction for developers of such replicated services. It is specifically designed around managing a linear, append-only log, which stores all **SMR** operations for the application. Because this log is identical across all replicas, executing the operations in the order determined by it ensures that every node arrives at the same final state.

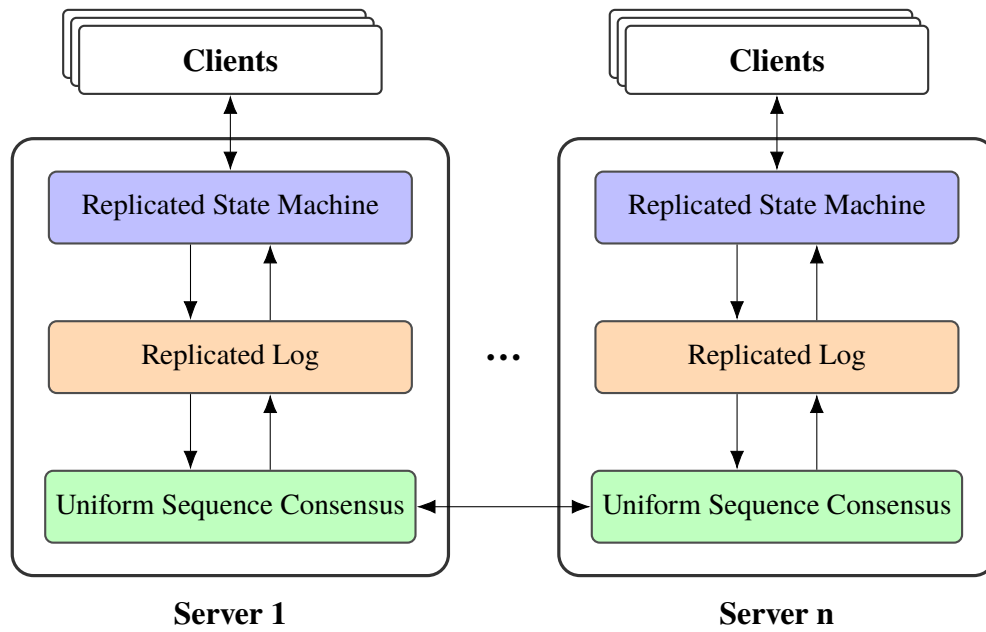


Figure 2.1: Architecture of a **SMR** service using a replicated log abstraction.

Figure 2.1 illustrates a common architecture of a fault-tolerant system, acting as a replicated state machine. External users and applications send requests to, ideally co-located, servers, which then try to append commands to a replicated log in case the requests should result in a change of state. As can be seen, the underlying challenge addressed by Raft and similar protocols is uniform sequence consensus, which provides a notion of strong consistency and is detailed in Section 2.1.1.

2.1.1 Uniform Sequence Consensus

As described, the problem of agreeing on a value between coordinating processes lies at the core of fault-tolerant distributed systems. Any algorithm that relies on building a common state or synchronizing an action in an environment subject to partial failures must, in some form, solve the consensus problem.

We can formalize the abstraction layer of *uniform consensus* according to Cachin et al. [24]. In this model, processes interact with the consensus primitive through two events: *proposing* a candidate value and eventually *deciding* on a single output. A process triggers the consensus by proposing a value, and after a coordination phase, the abstraction issues a decision,

which delivers the agreed (*decided*) value back, which marks the finality of the consensus instance. An algorithm for uniform consensus must satisfy the following properties:

Validity: Every decided value was previously proposed.

Integrity: No process decides more than once.

Uniform agreement: No two processes decide different values.

Termination: Every correct process eventually decides some value.

Since uniform consensus is impossible to achieve in a purely *asynchronous system* with no bounds on clock drift, transmission delay, and processing time [25], we consider networks like the Internet as *partially synchronous systems* throughout this thesis. This model assumes that while delays occur, there are sufficiently long stable periods for algorithms to make progress.

Standard uniform consensus only deals with a single decision, but a replicated log requires agreement on an ordered sequence. In an extension of this model, every participating process can propose values to the consensus layer, which eventually decides on a sequence of values. The relevant properties of this *Uniform Sequence Consensus* have been defined by Haridi et al. [26]:

Validity: Every value in a sequence decided by a process was previously proposed.

Integrity: If a process decides a value sequence, it is a strict prefix of every future decided sequence by that server.

Uniform agreement: If two processes decide sequences of values, one sequence is a prefix of the other.

Termination: If a value is proposed infinitely often by a correct process, then every correct process eventually decides on a sequence containing this value.

Ultimately, these properties provide the formal framework necessary to ensure that a distributed log remains a consistent single source of truth, allowing a collection of nodes to function as a unified, reliable service at the application level.

2.1.2 Leader-Based Sequence Paxos

Paxos denotes a family of algorithms designed to provide the guarantees of uniform (sequence) consensus. Its simplest form, often referred to as basic Paxos or single-value Paxos, was originally introduced by Leslie Lamport [4] and solves uniform consensus within a partially synchronous system. It ensures that even under concurrent proposals from multiple nodes, the system converges on a single decision that satisfies the validity, integrity, and agreement properties defined in Section 2.1.1. Understanding this mechanism is essential before addressing the complexities of maintaining a continuous, ordered log.

Paxos decomposes the consensus process into three distinct phases. In the *prepare phase*, a process acting as a *proposer* attempts to claim the right to propose a value. It selects a unique, monotonically increasing proposal number n and broadcasts a *prepare* message containing n . Upon receipt, an *acceptor* returns a *promise* to the proposer, committing to reject any values from proposers that initiated a round with a lower proposal number than n . Crucially, if the acceptor has already accepted a value from a previous round, it includes that value and its associated proposal number in its response, forcing the proposer to respect any existing progress.

Once a proposer has secured promises from a majority (*quorum*) of acceptors, it proceeds to the *accept phase*, where it tries to make the acceptors lock in its proposal. It sends an *accept* message to the promised acceptors, containing the proposal number n and a value v to be accepted. If the previous phase reported an existing value, the proposer must use the one with the highest proposal number; otherwise, it is free to use its own. An acceptor acknowledges this via an *accepted* message, provided it has not already promised to a higher proposal number than n before.

As soon as a quorum of acceptors has accepted the same value, it is considered *chosen*. At this stage, no other process can ever decide on a different value for this instance. Any participant that notices the value being chosen can notify the processes interested in the result (*learners*). This is often done by the original proposer by broadcasting a *decided* message to the learners.

An example execution trace of a Paxos instance is displayed in Figure 2.2 with a total of three acceptors, who simultaneously act as learners. The process $P1$ wants to propose a value and first initiates the prepare phase. After a majority quorum of acceptors ($P1$ and $P2$) has been formed using promises, $P1$ has gained the right to propose. In the accept phase, $P1$ gets a majority

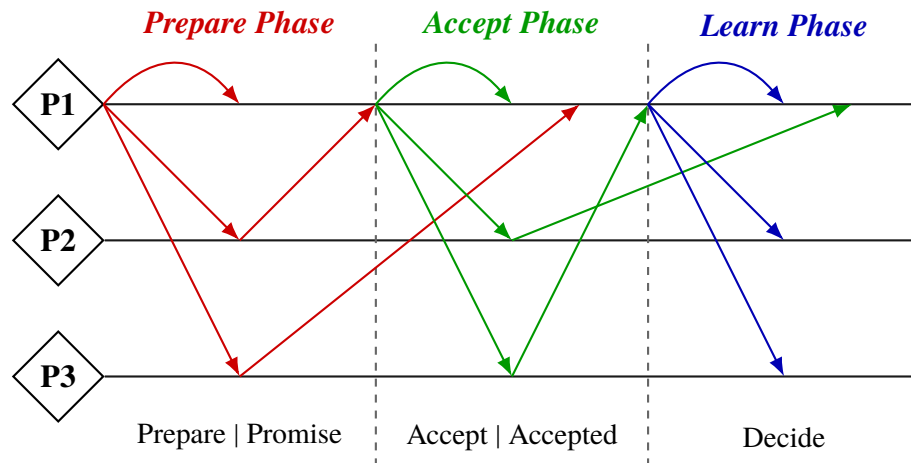


Figure 2.2: Example execution of Paxos with three servers.

quorum to lock in its value (*P1* and *P3*), recognizes the result being chosen, and broadcasts the decided value in the learn phase.

As concurrent proposers might take the right to propose from each other over and over again, thus stopping the system from making progress, a leader election can be used to elect a single distinguished proposer, who will eventually be the only process allowed to propose a value [4]. With such a mechanism in place, the termination property of uniform consensus is also satisfied. This principle also directly leads to *leader-based Sequence Paxos*, an extension of Paxos that handles an ordered sequence of values and provides the guarantees of uniform sequence consensus (see Section 2.1.1).

The primary optimization here is that if the leader remains stable, the prepare phase can be performed once for a whole series of log entries, rather than once per entry. This allows the system to effectively pipeline proposals through the accept phase using the elected leader as the distinguished proposer [4, 26, 27]. While specific implementations may augment the prepare phase with log-synchronization mechanisms depending on the leader election algorithm, this main idea stays the same across all implementations of uniform sequence consensus. Figure 2.3 displays a common execution of a leader-based sequence Paxos algorithm where *P1* now acts as the leader and pipelines values to be decided in a sequence one by one. The decided values are then free to be appended to a replicated log in this sequence.

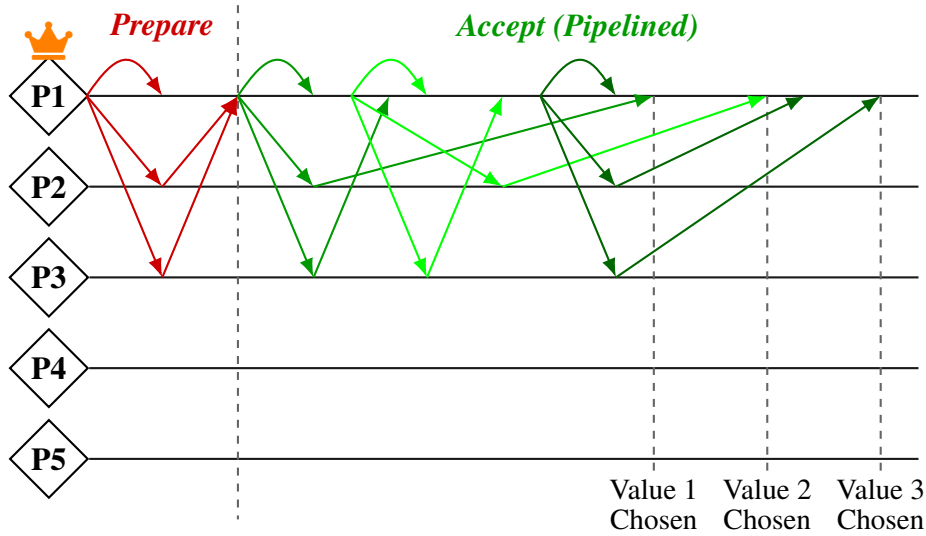


Figure 2.3: Example execution of leader-based Sequence Paxos with five servers. Chosen values can be broadcast to learners similar to before.

2.1.3 Fast-Path Integration

In the leader-based Sequence Paxos algorithm described above, only the leader is permitted to propose values for the log. Consequently, when a non-leader process wishes to propose a value, the operation incurs a total of three message delays [18]:

1. The original proposer forwards its value to the leader to initiate the append operation.
2. The leader sends accept messages to all promised acceptors.
3. The acceptors respond with corresponding accepted messages either directly to the learners or to the leader, who subsequently informs the learners once a value is chosen.

The primary motivation for introducing Fast Paxos is to reduce this latency by one message delay, allowing any participating process to propose values directly to the acceptors instead of first forwarding them to the leader. This optimization was originally devised by Leslie Lamport [5]. In such a *fast round*, acceptors simply accept the first value they receive from any proposer and notify the leader (and potentially the learners) of their decision via an *accepted* message.

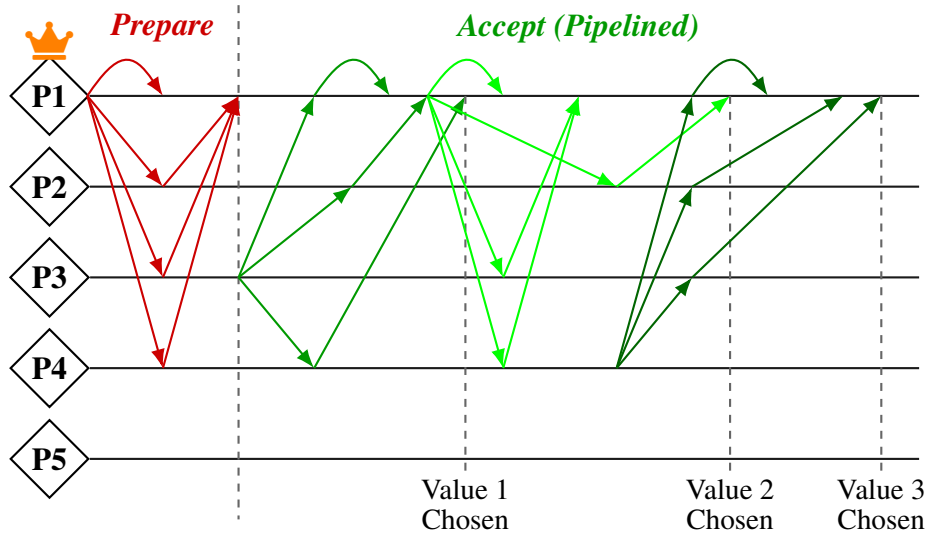


Figure 2.4: Example execution of leader-based Sequence Paxos with five servers using the fast path successfully.

Compared to the original algorithm discussed earlier, a simple majority quorum is no longer sufficient to consider a value chosen when using the fast path. To guarantee safety, the intersection of two quorums for a fast round (*fast quorums*) must include a node from any possible majority quorum [6]. In practice, the size of a fast quorum must be at least $\lceil \frac{3n}{4} \rceil$ for a system of n acceptors. This requirement also implies that the fast-path algorithm can tolerate fewer concurrent failures within a round compared to the standard design.

A typical message flow for a successful fast path is depicted in Figure 2.4. The leader ($P1$) is no longer the only one that proposes values in the sequence, but other nodes do so as well with the leader collecting the accepted messages. A value is now considered chosen once a fast quorum has accepted the same value for an index in the sequence, which is recognized by the leader.

The fast path is inherently optimistic, however. A collision can occur when multiple proposers submit different values simultaneously such that no single value achieves the fast quorum threshold. The leader can detect such collisions and initiate a regular Paxos round using the most frequently accepted value. This *slow path* recovery requires two additional communication steps, meaning that a fast round with a collision is even slower than a round following the original algorithm. Figure 2.5 shows such a collision case, where $P2$ and $P4$ make a concurrent proposal for the second value in the sequence. The collision is detected by the leader ($P1$) when no fast quorum can be formed,

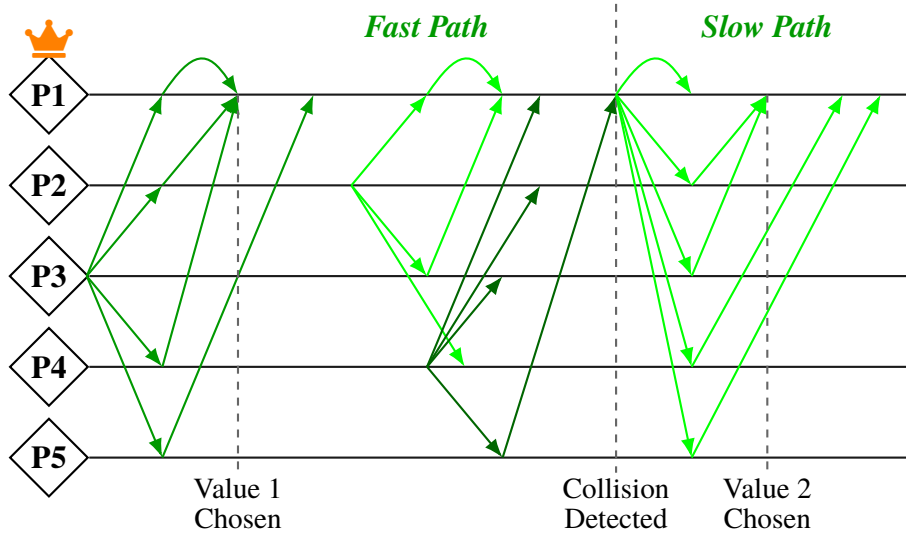


Figure 2.5: Example execution of leader-based Sequence Paxos with five servers and a collision of proposals.

resulting in *P1* initiating a slow path for the second slot, where a value is chosen as soon as a majority quorum accepts the value proposed by the leader.

2.2 Conformal Prediction

When deploying predictive models in high-stakes applications, evaluating prediction accuracy alone is insufficient. It is equally important to quantify the uncertainty of model predictions. This makes **Uncertainty Quantification (UQ)** a central research topic in statistics and **Machine Learning (ML)**.

Instead of producing a single-point prediction, one way to quantify predictive uncertainty is to output *prediction sets* (for classification) or *prediction intervals* (for regression). **Conformal Prediction (CP)** is a statistical framework for constructing such prediction sets or intervals with formal guarantees. The primary objective is to transform a heuristic notion of uncertainty from an underlying model into statistically rigorous prediction sets or intervals that contain the ground truth with a user-defined probability. **CP** has several desirable properties that make it suited for practical applications [19, 20, 28]:

- *Model-agnostic*: It can be applied on top of any predictive model without requiring changes to the model architecture. **CP** fundamentally treats the underlying model as a black box.

- *Distribution-free*: Valid coverage is provided under the assumption of data exchangeability. Exchangeability means the data points can be reordered without changing their joint distribution. Informally, no observation carries information through its position in the dataset.
- *Finite-sample valid*: Coverage is guaranteed even if prediction is constructed based on a finite number of data points.

In practice, conformal methods must satisfy additional requirements beyond theoretical validity. In particular, computational efficiency is important. Ideally, **CP** should function as a lightweight wrapper around an existing model without requiring retraining. Another important consideration is the *informativeness* of the result [28]. While coverage is guaranteed, the practical usefulness of conformal prediction depends on producing sets or intervals that remain small when the model is confident and expand only when uncertainty is high. They should provide actionable insights about the model's output.

At the core of the conformal framework is the *non-conformity score*, which quantifies the inconsistency of a labeled example where larger values indicate that the label poorly matches the feature. The function itself is user-defined and describes the heuristic notion of uncertainty. The choice of function primarily determines the informativeness of the resulting prediction sets.

The following sections introduce specific conformal methods and describe how they can be used for quantifying uncertainty in classification tasks.

2.2.1 Full Conformal Prediction

The central idea behind Full **CP** for classification is to evaluate how plausible each possible label would be if it were assigned to a new test example. Rather than committing to a single prediction, the method tests every candidate label and retains those that appear consistent with seen data. The collection of all labels that pass this test forms the prediction set returned by the method. What consistent means in this context depends on the user-specified probabilistic coverage guarantee the prediction set has to fulfill.

Formally, consider a training dataset consisting of feature-label pairs $(X_1, Y_1), (X_2, Y_2), \dots, (X_n, Y_n)$ and a test point $(X_{test}, Y_{test}) \in \mathcal{X} \times \mathcal{Y}$ with the test label being unobserved. The aim is to construct a prediction set for the test feature $\mathcal{C}(X_{test}) \subseteq \mathcal{Y}$ such that it satisfies the *marginal predictive coverage* for an error rate (or *significance level*) $\alpha \in [0, 1]$ defined as

Algorithm 2.1 Full Conformal Prediction for Classification**Inputs:**Training data $\mathcal{D} = \{(X_1, Y_1), \dots, (X_n, Y_n)\} \in (\mathcal{X} \times \mathcal{Y})^n$ Test feature $X_{\text{test}} \in \mathcal{X}$ Target error rate $\alpha \in [0, 1]$ Non-conformity score $s : (\mathcal{X} \times \mathcal{Y}) \times (\mathcal{X} \times \mathcal{Y})^* \rightarrow \mathbb{R}$ **Output:**Prediction set $\mathcal{C}(X_{\text{test}})$

```

1: for  $y \in \mathcal{Y}$  do
2:   Augmented training set:  $\mathcal{D}^y \leftarrow \mathcal{D} \cup (X_{\text{test}}, y)$ 
3:   Conformal scores:  $S_i^y \leftarrow s((X_i, Y_i); \mathcal{D}^y)$  for all  $i \in \{1, \dots, n\}$ 
4:   Conformal score of hypothesized test point:  $S_{\text{test}}^y \leftarrow s((X_{\text{test}}, y); \mathcal{D}^y)$ 
5:   Quantile:  $\hat{q}^y \leftarrow \text{Quantile}(S_1^y, \dots, S_n^y, S_{\text{test}}^y; (1 - \alpha)(1 + 1/n))$ 
6: end for
7: return  $\mathcal{C}(X_{\text{test}}) \leftarrow \{y \in \mathcal{Y} : S_{\text{test}}^y \leq \hat{q}^y\}$ 

```

follows [20, 28, 29]:

$$\mathbb{P}(Y_{\text{test}} \in \mathcal{C}(X_{\text{test}})) \geq 1 - \alpha$$

In other words, the prediction set should contain the true label with probability at least $1 - \alpha$ when evaluated over new data.

In *full CP*, the non-conformity score $s((x, y); \mathcal{D}) \in \mathbb{R}$ measures the error of a model trained with $\mathcal{D} \in (\mathcal{X} \times \mathcal{Y})^*$ on the feature-label pair $(x, y) \in \mathcal{X} \times \mathcal{Y}$ [20]. For classification, a common score function is $s((x, y); \mathcal{D}) = 1 - \hat{p}_y(x)$ where $\hat{p}_y(x)$ denotes the predicted probability of class y when plugging in feature x into a model trained using $\mathcal{D}$.

The core idea of *full CP* is to systematically test every candidate label y in the entire label space $\mathcal{Y}$ for its plausibility relative to the observed training data. For each candidate label, *full CP* constructs a hypothetical augmented dataset by pairing the test feature X_{test} with that specific candidate and treating this pair as a legitimate part of the original training set. The non-conformity score is computed for every point in this new, larger dataset, which involves a complete training cycle of the underlying model for every possible label. If the score of the hypothesized test point (X_{test}, y) is higher than the $(1 - \alpha)$ quantile of the other scores, the label is deemed inconsistent. Algorithm 2.1 shows the full conformal prediction process as adapted from Angelopoulos et al. [29], which also includes finite-sample corrections in the quantile computation.

Algorithm 2.2 Split Conformal Prediction for Classification

Inputs:Model fitting data $\mathcal{D}_{mod} \in (\mathcal{X} \times \mathcal{Y})^m$ Calibration data $\mathcal{D}_{cal} = \{(X_1, Y_1), \dots, (X_n, Y_n)\} \in (\mathcal{X} \times \mathcal{Y})^n$ Test feature $X_{test} \in \mathcal{X}$ Target error rate $\alpha \in [0, 1]$ **Output:**Prediction set $\mathcal{C}(X_{test})$

-
- 1: Construct a score function $s : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$ using $\mathcal{D}_{mod}$
 - 2: Conformal scores of calibration set: $S_i \leftarrow s(X_i, Y_i)$, for all $i \in \{1, \dots, n\}$
 - 3: Conformal quantile: $\hat{q} \leftarrow \text{Quantile}(S_1, \dots, S_n; (1 - \alpha)(1 + 1/n))$
 - 4: **return** $\mathcal{C}(X_{test}) \leftarrow \{y \in \mathcal{Y} : s(X_{test}, y) \leq \hat{q}\}$
-

Intuitively, when performing the test with the correct label, the test point becomes statistically indistinguishable from the training data, meaning its score is unlikely to be very high. By only rejecting labels that make the test point look like a rare anomaly compared to the rest of the group, it is ensured that the true label is retained with the exact probability specified.

2.2.2 Split Conformal Prediction

Full **CP** as described is computationally impractical. It requires full retraining for each possible label and a repetition of this process for every new observation. A computationally efficient variant called *split CP* instead divides the training set into data used for model fitting $\mathcal{D}_{mod} \in (\mathcal{X} \times \mathcal{Y})^m$ and data used as a calibration set $\mathcal{D}_{cal} \in (\mathcal{X} \times \mathcal{Y})^n$ for computing the non-conformity scores as a comparison [30]. Algorithm 2.2 shows the updated mechanism for split **CP**.

The fundamental shift in split **CP** is the freezing of the underlying predictive model trained on $\mathcal{D}_{mod}$ from which a non-conformity score $s : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$ can be constructed. Compared to the full conformal setting from before, $s(x, y)$ becomes a fixed mapping that no longer changes when we evaluate on features and labels not involved in the model creation process. This decoupling is what allows split **CP** to bypass the exhaustive retraining required by the full version [29]. Note that this approach also allows direct construction of such a score function using, for instance, domain-specific knowledge to capture a heuristic notion of uncertainty.

The validity of split **CP** relies on the assumption that the calibration data

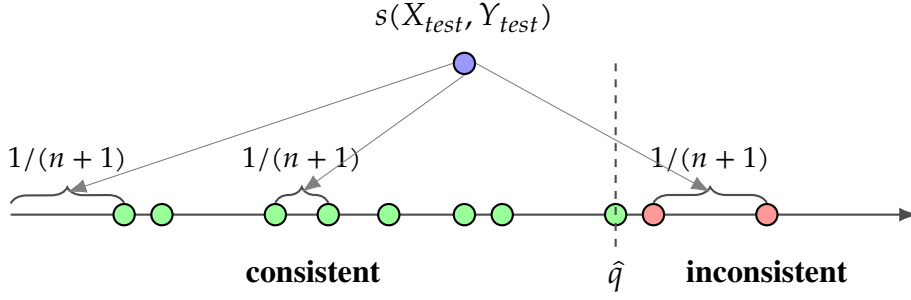


Figure 2.6: A depiction of split conformal prediction with a calibration set. The score of a new test feature has an equal probability to fall in between any two calibration scores when the correct label Y_{test} is plugged in. This actual score will be greater than the consistency threshold and therefore inconsistent with the calibration data with a probability of α .

$\mathcal{D}_{cal}$ and any test point (X_{test}, Y_{test}) are exchangeable. If this holds and none of the data points were used for model construction, the corresponding non-conformity scores $\{S_1, \dots, S_n, S_{test}\}$ are also exchangeable. This ensures that the non-conformity score of a previously unseen test observation is equally likely to occupy any rank when compared against a set of calibration scores [20], as shown in Figure 2.6. When we therefore include all possible labels $y \in \mathcal{Y}$ in the final prediction set, where the scores $s(X_{test}, y)$ end up in the $1 - \alpha$ quantile of the calibration scores, we guarantee that it contains the ground truth with a probability of $1 - \alpha$.

Since the calibration set $\mathcal{D}_{cal}$ is fixed during the deployment phase, the consistency threshold $\hat{q}$ can be precomputed once. For any incoming test feature X_{test} , generating the prediction set $\mathcal{C}(X_{test})$ simply requires computing the non-conformity scores for every possible label (which does not require retraining) and a comparison of the resulting scores against the precomputed $\hat{q}$.

Split CP in the presented form also has a desirable upper bound on the coverage with a calibration set of size n .

$$1 - \alpha \leq \mathbb{P}(Y_{test} \in \mathcal{C}(X_{test})) \leq 1 - \alpha + \frac{1}{n+1}$$

2.2.3 Conformal Risk Control

While classical CP guarantees that the true label is contained in the prediction set with probability at least $1 - \alpha$, this notion of coverage does not always align

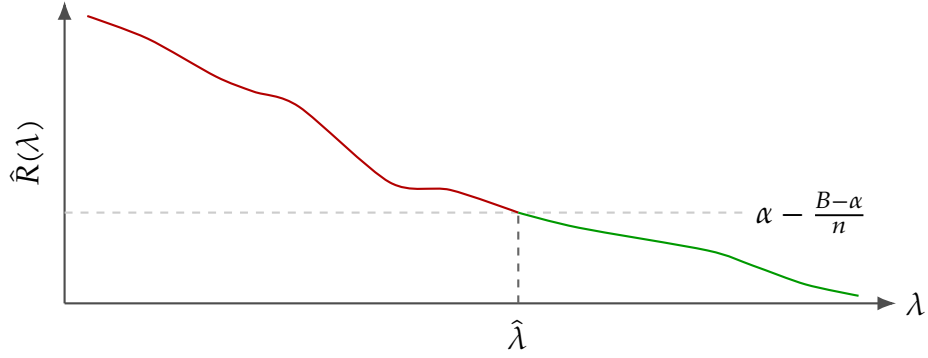


Figure 2.7: Empirical risk $\hat{R}(\lambda)$ as a function of the tuning parameter λ adapted from [29]. The selected value $\hat{\lambda}$ is the smallest parameter for which the empirical risk falls below the target risk level.

with the operational objectives of a deployed system. Many applications care about controlling a specific *risk* associated with predictions rather than the simple probability of excluding the true label. For instance, a goal could be to limit the **False Negative Rate (FNR)** for a specific class or control the error rate of an out-of-distribution detector. **Conformal Risk Control (CRC)** therefore extends the conformal prediction framework to address such objectives by providing guarantees on user-defined risk measures [21, 29].

The central idea of **CRC** for classification is again to construct a prediction set $C_\lambda(X_{test})$ for a test point $(X_{test}, Y_{test}) \in \mathcal{X} \times \mathcal{Y}$ with an unknown label while given a calibration dataset of feature-label pairs $(X_1, Y_1), (X_2, Y_2), \dots, (X_n, Y_n)$. Unlike classical **CP**, which guarantees that the true label is contained in the set with a fixed probability, **CRC** provides an expectation bound on a user-defined loss function $\mathcal{L}$ [21]. For a bounded loss $\mathcal{L} : 2^{\mathcal{Y}} \times \mathcal{Y} \rightarrow (-\infty, B]$ and a desired risk limit $\alpha \in (-\infty, B]$, **CRC** ensures:

$$\mathbb{E}(\mathcal{L}(C_{\hat{\lambda}}(X_{test}), Y_{test})) \leq \alpha$$

The mechanism for achieving this guarantee is the parameter λ , which acts as a threshold for set inclusion. Instead of calculating a single quantile as in classical **CP**, **CRC** searches over a family of prediction rules $C_\lambda(x)$. The validity of the framework relies on two properties of monotonicity [29]:

1. As λ increases, the prediction set $C_\lambda(x)$ must grow or stay the same size:

$$\lambda_1 \leq \lambda_2 \Rightarrow C_{\lambda_1}(x) \subseteq C_{\lambda_2}(x)$$

2. Larger prediction sets must result in a lower (or equal) loss:

$$C \subseteq C' \Rightarrow \mathcal{L}(C, y) \geq \mathcal{L}(C', y)$$

By combining these properties, the empirical risk $\widehat{R}(\lambda)$ defined as the average loss over the calibration set becomes a monotonically decreasing function of λ :

$$\widehat{R}(\lambda) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(C_\lambda(X_i), Y_i)$$

In a classification setting, a common prediction set construction method based on $\lambda \in [0, 1]$ can look like this:

$$C_\lambda(x) = \{y : \hat{p}_y(x) \geq \lambda\}$$

$\hat{p}_y(x)$ again denotes the probability of class y when plugging in feature x into some predictive model, which means that the prediction set includes all classes where this predicted probability is larger than λ .

The objective is to select the smallest value of λ whose empirical risk estimate remains below the desired risk limit α , as shown in Figure 2.7. With the inclusion of correction terms due to sampling variability in the risk estimate, this equation yields a valid selection rule for $\hat{\lambda}$ [21, 29]:

$$\hat{\lambda} = \inf \left\{ \lambda : \frac{n}{n+1} \widehat{R}(\lambda) + \frac{B}{n+1} \leq \alpha \right\} = \inf \left\{ \lambda : \widehat{R}(\lambda) \leq \alpha - \frac{B - \alpha}{n} \right\}$$

Algorithm 2.3 formalizes the utilization of conformal risk control in a classification setting. In a practical implementation, $\hat{\lambda}$ can be efficiently pre-computed using a basic root-search mechanism.

2.2.4 Adaptive Conformal Inference

The conformal prediction methods discussed thus far rely heavily on the assumption of data exchangeability. However, in many real-world online learning or time-series applications, the underlying data-generating distribution shifts over time [31, 32, 33]. When this exchangeability assumption breaks down, the theoretical coverage and risk guarantees of standard conformal prediction no longer hold.

To address this limitation, Gibbs and Candès introduced **Adaptive Conformal Inference (ACI)**, which adapts to arbitrary distribution shifts by

Algorithm 2.3 Conformal Risk Control for Classification

Inputs:

Model fitting data $\mathcal{D}_{mod} \in (\mathcal{X} \times \mathcal{Y})^m$
 Calibration data $\mathcal{D}_{cal} = \{(X_1, Y_1), \dots, (X_n, Y_n)\} \in (\mathcal{X} \times \mathcal{Y})^n$
 Test feature $X_{test} \in \mathcal{X}$
 Loss function $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \rightarrow (-\infty, B]$
 Risk limit $\alpha \in (-\infty, B]$

Output:

Prediction set $\mathcal{C}(X_{test})$

- 1: Get construction method for prediction sets $\mathcal{C}_\lambda(x)$ using $\mathcal{D}_{mod}$
 - 2: **for** $\lambda \in \Lambda$ **do**
 - 3: Calibration losses: $L_i^\lambda \leftarrow \mathcal{L}(\mathcal{C}_\lambda(X_i), Y_i)$ for $i = 1, \dots, n$
 - 4: Empirical risk: $\hat{R}^\lambda \leftarrow \frac{1}{n} \sum_{i=1}^n L_i^\lambda$
 - 5: **end for**
 - 6: Select parameter: $\hat{\lambda} \leftarrow \inf \left\{ \lambda : \frac{n}{n+1} \hat{R}^\lambda + \frac{B}{n+1} \leq \alpha \right\}$
 - 7: **return** $\mathcal{C}_{\hat{\lambda}}(X_{test})$
-

treating the significance level α as a dynamic parameter that is continuously updated based on the model's past errors [31]. Feldman et al. subsequently extended this concept to accommodate generic user-defined loss functions, known as *Rolling Risk Control* [33].

Rather than using a fixed calibration set to compute a static threshold, Rolling Risk Control updates the threshold parameter λ_t at each time step t . Upon receiving a new feature X_t , the system outputs a prediction set $\mathcal{C}_{\lambda_t}(X_t)$. Once the true label Y_t is revealed, the loss $\mathcal{L}(\mathcal{C}_{\lambda_t}(X_t), Y_t)$ is evaluated, and the parameter is updated for the next time step using a fixed learning rate $\gamma > 0$:

$$\lambda_{t+1} = \lambda_t + \gamma (\mathcal{L}(\mathcal{C}_{\lambda_t}(X_t), Y_t) - \alpha)$$

If the model's recent empirical risk exceeds the target limit α , the parameter λ_t is increased to produce more conservative (larger) prediction sets in subsequent steps. Conversely, if the risk falls below α , the sets are made smaller by a decrease in λ_t .

For this procedure to be valid, the set construction function must satisfy the monotonicity properties detailed in Section 2.2.3. Furthermore, it is assumed that the loss function is bounded, and that extreme values of λ guarantee specific bounds. Producing the full label space $\mathcal{Y}$ strictly satisfies the target risk ($\mathcal{L}(\mathcal{Y}, y) < \alpha$), and producing the empty set $\emptyset$ strictly violates

Algorithm 2.4 Rolling Risk Control for Classification

Inputs:Model fitting data $\mathcal{D}_{mod} \in (\mathcal{X} \times \mathcal{Y})^m$ Inference data (stream) $\mathcal{D} = \{(X_1, Y_1), \dots, (X_T, Y_T)\} \in (\mathcal{X} \times \mathcal{Y})^T$ Loss function $\mathcal{L} : 2^{\mathcal{Y}} \times \mathcal{Y} \rightarrow (-\infty, B]$ Risk limit $\alpha \in (-\infty, B]$ Learning rate $\gamma \in \mathbb{R}^+$ **Output:**Prediction sets (stream) $\{C(X_1), \dots, C(X_T)\}$

-
- 1: Get construction method for prediction sets $C_\lambda(x)$ using $\mathcal{D}_{mod}$
 - 2: Initialize $\lambda_1 \leftarrow 0$
 - 3: **for** $t \in \{1, \dots, T\}$ **do**
 - 4: Obtain feature vector X_t
 - 5: Output prediction set: $C_{\lambda_t}(X_t)$
 - 6: Obtain true label Y_t
 - 7: Loss: $L_t \leftarrow \mathcal{L}(C_{\lambda_t}(X_t), Y_t)$
 - 8: Update $\lambda_{t+1} \leftarrow \lambda_t + \gamma(L_t - \alpha)$
 - 9: **end for**
-

it $(\mathcal{L}(\emptyset, y) > \alpha)$ for all $y \in \mathcal{Y}$ [33]. Under these conditions, Rolling Risk Control provides a robust, distribution-free guarantee that the time-averaged empirical risk converges exactly to the target risk limit α :

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathcal{L}(C_{\lambda_t}(X_t), Y_t) = \alpha$$

Algorithm 2.4 details the operational deployment of Rolling Risk Control for classification tasks. Later, Section 3.3 shows how CRC and Rolling Risk Control can be applied to the problem addressed in this thesis.

2.3 Related Work

To ground this work, we examine three intersecting areas: modern fast-path consensus protocols, adaptive mechanisms for consensus under varying conditions, and the emerging use of conformal methods for data systems.

2.3.1 Modern Fast-Path Consensus Protocols

Although Fast Paxos can be used to implement a replicated log, doing so limits the effectiveness of its fast path. As a result, subsequent work often treats ordering as a flexible property and applies coordination only where required to preserve the correctness of the application-level state machine.

Generalized Paxos [34] first formalized agreement on partially ordered command structures, enabling fast decisions when operations commute. EPaxos [15] demonstrates how this idea can be realized in practice by replacing total order with dependency tracking, allowing replicas to execute independent commands without global coordination. Crucially, these relaxed guarantees make it possible to use the fast path with a low collision rate, since in practice most operations do not conflict.

Several protocols extend this line of work under the weakened conditions, including Caesar [35], Atlas [36], BPaxos [37], Skyros [38], and Gryff [39]. These systems explore different mechanisms for ordering, conflict detection, and system structuring, while retaining an optimistic fast path with a fallback when conflicts occur. Curp [16] allows clients to reach durability for non-conflicting requests by replicating them to a set of witnesses, deferring the more expensive task of ordering to the background when conflicts arise.

Across these designs, a common pattern emerges: modern fast-path consensus protocols often avoid the abstraction of a totally ordered replicated log and instead rely on partial orders and dependency tracking to reduce coordination. However, this shift has limitations, as log-free or partially ordered approaches are not universally applicable and may not provide the guarantees required by all applications.

Nezha [17] takes a different approach by attempting to make the fast path viable while retaining a replicated log. It does so by forcing replicas to observe requests in the same order using a deadline-ordered multicast. This reduces collisions through network timing control.

2.3.2 Adaptive Consensus

While consensus protocols are typically engineered as fixed, one-size-fits-all designs, real-world deployments operate under highly variable conditions, motivating research into adaptive mechanisms that adjust protocol behavior at runtime.

Junqueira et al. [18] show, both analytically and through simulation, that in wide-area deployments with high variance in message latency, Fast Paxos can in fact be slower than Classic Paxos with significant probability, despite

its theoretically shorter message path. This result highlights that the relative performance of consensus protocols depends strongly on network conditions, quorum sizes, and message latency variance.

Howard et al. [40] show that it is sufficient for quorums from different phases to intersect, allowing phase quorums to be chosen independently and enabling a wider range of quorum configurations. This flexibility allows systems to tune quorum sizes to trade off latency, throughput, and fault tolerance depending on deployment conditions.

More recent work explores adaptive behavior more directly. Reginald et al. [41] propose modeling failures probabilistically using machine-specific failure probabilities rather than a fixed failure threshold. Shiozaki et al. [42] explore dynamic tuning of Raft’s election parameters based on network conditions. Hu et al. [43] present Crossword, an adaptive consensus protocol designed for dynamic, data-heavy workloads.

2.3.3 Conformal Methods for Data Systems

Applying the principles of CP to data management systems has only recently begun to receive attention in research. Li et al. [44] introduced dbET, which constructs execution time distributions for query plans of database management systems. They use conformal methods to produce intervals that are guaranteed to cover the actual execution time with a predefined probability level, which can then be used for query optimization.

Liu et al. [45] employ CP to provide rigorous statistical guarantees and runtime verification for learned query optimizers, addressing their potential for performance instability. They formulate the verification as a CP problem to derive user-controlled bounded ranges for actual query latency before execution, ensuring that plans meet performance constraints at a specified confidence level. They also discuss adaptive methods for handling distribution shift.

Finally, Horchidan et al. [46] present ConANN, the first framework to provide formal, distribution-free error guarantees using CRC for approximate nearest neighbor search in an inverted file vector index. The system dynamically calibrates the number of clusters probed for each query, ensuring that the FNR remains below a user-defined threshold with high confidence.

2.4 Summary

We presented the theoretical foundations for both replicated state machines and uncertainty-aware decision-making. **SMR** was described as the standard approach for building fault-tolerant services, with uniform sequence consensus and leader-based Sequence Paxos providing the formal and practical basis for maintaining a replicated log. Fast Paxos and related fast-path techniques reduce latency by allowing replicas to bypass the leader, but this optimization is inherently optimistic: it reduces message delays when collisions are rare but can degrade performance when contention is high.

As methods to address uncertainty, we introduced **CP** and **CRC** as statistical frameworks that provide formal guarantees. Split conformal prediction enables efficient uncertainty estimation in practical systems, while **CRC** allows guarantees over system-level risk rather than simple prediction coverage. These methods make it possible to transform heuristic confidence estimates into decisions with formal guarantees and can be extended to an online setting that involves time-series data.

Finally, we highlighted the gap in prior work where adaptive mechanisms for replicated logs remain limited while conformal methods have been successfully applied to provide statistical guarantees for data management systems. This motivates the approach of this thesis: using uncertainty-aware methods to adapt the use of fast-path consensus in replicated log systems.

Chapter 3

Conformal Consensus

This chapter introduces an adaptive, decentralized architecture for fast-path routing in a replicated log abstraction layer with statistical guarantees on fast-path risk. Whenever a node attempts to append a proposal, it evaluates real-time conditions to determine the optimal operating mode (Regular Paxos or Fast Paxos) through an internal predictor. With nodes being able to select modes independently, the goal is to enable the system to handle heterogeneous workloads in a way that reduces end-to-end latency while maintaining trust through conformal guarantees.

The coexistence of multiple operating modes for the same log slot within a single ballot requires interoperability mechanisms. The underlying consensus layer must be modified to evaluate mixed-mode proposals and detect and resolve collisions without compromising the safety or liveness of replicated logs. By calibrating a dynamic selection threshold using **CRC** and Rolling Risk Control, the design controls the long-term empirical rate of fast-path failures toward a user-defined target, safeguarding the system against the latency penalties of frequent slow-path recoveries.

Section 3.1 describes consensus modifications required to evaluate mixed-mode proposals and resolve collisions. Section 3.2 establishes a heuristic for estimating fast-path success based on real-time metrics. Finally, Section 3.3 details how this heuristic is integrated into a risk-controlled prediction layer to provide a statistical guarantee on fast-path failures.

3.1 Decentralized Mode Switching

To achieve a decentralized architecture where nodes evaluate operating modes independently, the consensus protocol must account for structural

asymmetries within the cluster. While followers actively select between forwarding proposals (Regular Paxos) or initiating localized fast-path broadcasts (Fast Paxos), the leader consistently operates in the regular mode. This asymmetrical design stems from two practical considerations: a standard majority quorum is mathematically easier to achieve than a larger fast quorum, and the leader inherently avoids the forwarding latency penalty that represents the primary drawback of regular mode execution. Allowing individual followers to make localized decisions, however, requires specialized interoperability rules (see Section 3.1.1) and modified leader-side tracking logic (see Section 3.1.2) to maintain consistency.

As noted previously, the purpose of the protocol modifications presented in this section is not to design a production-ready consensus algorithm. Instead, they define an experimental protocol variant that is sufficiently specified to study the feasibility and performance implications of decentralized mode switching under controlled assumptions. Consequently, several aspects required for a complete consensus protocol, including leader election, recovery after failures, persistent state, and full correctness and liveness proofs, are intentionally excluded from the scope of this work. Instead, we focus merely on scenarios with a stable leader, which is connected to at least a fast quorum of nodes.

3.1.1 Mode Interoperability

The coexistence of multiple operating modes for the same slot within a single ballot necessitates a hybrid design that allows message processing from different protocol variants without compromising safety. To achieve this interoperability, every `Accept` and `Accepted` message is tagged with a corresponding `AcceptStatus`:

- **LeaderAccept (Regular Path):** Indicates a proposal issued by the leader via the standard Paxos protocol.
- **FastAccept (Fast Path):** Indicates a proposal initiated directly by a follower attempting to reach a fast quorum.
- **SlowAccept (Slow Path):** Indicates a remedial proposal issued by the leader to resolve a detected collision of proposals.

Append handling depends on the node's local mode selection. In *Fast Paxos* mode, the node broadcasts an `Accept` message with the `FastAccept` status directly to peers after assigning the slot with the smallest index that is

still empty to the proposal. In *Regular Paxos* mode, the proposal is forwarded to the current leader, which assigns the slot and broadcasts an `Accept` message with the `LeaderAccept` status.

A challenge in decentralized mode switching is a collision, where multiple nodes attempt to occupy the same log slot with different values via the fast path or the regular path. To maintain liveness, the leader maintains a record of all `Accepted` messages received from the cluster for every open slot. If the leader detects that no entry can be decided on due to a lack of responses or conflicting values for the same index, it triggers a resolution via the slow path. The concrete logic for the collision detection is described in Section 3.1.2. When having to use the slow path, the leader determines the correct entry to choose for this slot and broadcasts a new `Accept` message with the `SlowAccept` status.

The safety of the replicated log relies on strict rules regarding when a node is allowed to update its local state. `FastAccept` and `LeaderAccept` messages are treated as first-come, first-served. A follower is only allowed to accept these entries if the corresponding slot in its local log is currently empty. This rule is intended to prevent an uncoordinated follower from overwriting a value that may already have achieved a quorum elsewhere. In contrast, a message with `SlowAccept` status is treated as a command to resolve a conflict. Since this status is only issued by the leader after observing the cluster state, it is permitted to override any previously accepted value in a follower's log. Under the assumption of a stable leader and fast-quorum connectivity, this hierarchy is intended to make the leader converge conflicting proposals toward a single value.

3.1.2 Quorum Logic and Collision Detection

This section details the deterministic logic utilized by the cluster leader to evaluate quorums, detect proposal collisions, and drive individual log slots to a decided state. None of the additional state maintained by the leader requires persistence to stable storage for recovery purposes even if applied in a fail-recovery model.

To evaluate the current state of an uncommitted log entry, the leader leverages Algorithm 3.1. This algorithm operates as a pure function that takes the set of collected proposals and votes for a given slot and returns its discrete operational status: `PENDING`, `COLLISION(entry)` with `entry` being a safe value to execute a slow path with or `CHOSEN(entry)` where the `entry` is free to be learned.

Algorithm 3.1 Quorum Evaluation and Collision Resolution

```

1: procedure COMPUTEPROPOSERESULT(proposals)
2:   if not majority quorum on  $|proposals|$  then
3:     return PENDING
4:   end if
5:    $(e^*, leader, fast, slow) \leftarrow$  entry in proposals with maximum total votes
6:   if majority quorum on leader or
       fast quorum on fast or
       majority quorum on slow then
7:     return CHOSEN( $e^*$ )
8:   end if
9:   if not fast quorum on  $|proposals|$  then
10:    return PENDING
11:  end if
12:   $remaining \leftarrow |nodes| - |proposals|$ 
13:  for each  $(e, leader, fast, slow)$  in vote counts of proposals do
14:    if fast quorum on  $remaining + fast$  or
       majority quorum on  $remaining + leader$  or
        $slow > 0$  then
15:      return COLLISION( $e$ )
16:    end if
17:  end for
18:  return COLLISION( $e^*$ )
19: end procedure

```

The evaluation begins by checking for immediate quorum satisfaction. A value is chosen once it reaches the quorum required by its `AcceptStatus`. This is a standard majority quorum for entries tagged with `LEADERACCEPT` or `SLOWACCEPT`, or a larger fast quorum for entries tagged with `FASTACCEPT`. Because all these ways of acceptance require at least a majority of votes, it is enough to check the entry with the maximum total votes. Additionally, if less than a majority has voted, this also means that no decision could have been reached, making the slot `PENDING`.

If no single entry has achieved its required quorum, the leader must determine whether a collision has occurred and select a value that cannot conflict with a potentially chosen value for the slow path under the made assumptions. Merely collecting a majority of total cluster responses is insufficient to safely trigger collision resolution in this mixed-mode environment. Consider, for instance, a cluster of five nodes, where a regular

majority quorum consists of three and a fast quorum of four nodes. If the leader received three responses for a slot, with one being a `LEADERACCEPT` of entry *A* and two being `FASTACCEPT` messages of entry *B*, the outcome remains ambiguous, although a simple majority of responses has been gathered. With two nodes yet to respond, value *A* could still theoretically achieve a majority quorum, while value *B* could concurrently reach a fast quorum. Initiating a slow-path recovery at this stage risks overriding a value that might have achieved a quorum elsewhere in the network.

To avoid that, the leader must wait until a fast quorum of total responses has been collected for the slot. Then, it becomes mathematically impossible for more than one value to be able to reach its respective quorum using the remaining uncast votes. A proof of this is written out in Appendix A.

After this threshold is met, the leader executes the final loop of Algorithm 3.1 to identify a valid candidate for the slow path. The leader computes the number of uncast votes (*remaining*) and checks if any entry can still mathematically achieve its required threshold. If an entry retains the potential to reach its quorum, it is selected as the safe value for the slow path. In cases where high contention prevents any entry from being able to reach a quorum, the system falls back to the entry with the highest current vote count.

To translate the described quorum evaluation into concrete leader actions, the leader executes Algorithm 3.2 upon receiving any accepted proposal, whether initiated by a follower or by itself. This mechanism serves as a stateful wrapper around the functional quorum computation.

The procedure begins with an initial safeguard step: any proposal targeting a log index prior to the currently contiguous decided sequence (denoted as *decidedIdx*) is immediately discarded as obsolete. Valid proposals are subsequently added to the internal metadata for the corresponding slot, ensuring an up-to-date record of all cast votes. The algorithm then invokes `ComputeProposeResult` and maps its returned state to one of three specific leader actions:

- **DECIDEDACTION:** This action is triggered when the slot achieves a `CHOSEN` state and perfectly aligns with the current *decidedIdx*. Upon this trigger, the algorithm sweeps forward through the internal metadata to identify all consecutively chosen slots. These contiguous entries form a single suffix that can now be permanently decided, subsequently advancing the cluster's *decidedIdx*.
- **SLOWPATHACTION:** If the evaluation transitions the slot's state from `PENDING` to `COLLISION(e)`, the leader emits a `SLOWPATHACTION`. This

Algorithm 3.2 Leader Proposal Processing

```

1: procedure ADDNEWPROPOSAL(pid, slotIdx, entry, status)
2:   if slotIdx < decidedIdx then
3:     return NoACTION
4:   end if
5:   Append (pid, entry, status) to acceptMeta[slotIdx]
6:   previous ← slotResults[slotIdx]
7:   result ← COMPUTEPROPOSERESULT(acceptMeta[slotIdx])
8:   slotResults[slotIdx] ← result
9:   if result = CHOSEN(⌊) and slotIdx = decidedIdx then
10:    newDecidedSuffix ← [ ]
11:    while slotResults[decidedIdx] = CHOSEN(e) do
12:      Append (decidedIdx, e) to newDecidedSuffix
13:      Remove slotResults[decidedIdx]
14:      Remove acceptMeta[decidedIdx]
15:      decidedIdx ← decidedIdx + 1
16:    end while
17:    return DECIDEDACTION(newDecidedSuffix, decidedIdx)
18:   else if result = COLLISION(e) and previous = PENDING then
19:     return SLOWPATHACTION(slotIdx, e)
20:   else
21:     return NoACTION
22:   end if
23: end procedure

```

directive instructs the leader to broadcast an `Accept` message with the `SlowAccept` status for value e , thereby resolving the detected collision according to the hierarchical rules established in Section 3.1.1.

- **NoACTION**: This fallback is returned either when the slot remains in a `PENDING` state due to insufficient votes or when a collision has already been detected and acted upon in a previous iteration. The latter condition is required to prevent the emission of duplicate slow-path broadcasts for the same log index.

Entries, which have not been immediately decided or used for the slow path, can be directly retried for a different slot by the leader.

Assuming a stable leader and continued fast-quorum connectivity, the design is expected to preserve liveness during the accept phase. Compared to regular Sequence Paxos, if the cluster degrades to a simple majority, the

leader might not be able to safely resolve collisions, effectively halting the log.

3.2 Heuristic Score for Fast Path Success

While the leader-side proposal processing and collision resolution described previously attempts to provide safety and liveness, the followers must individually choose the operating modes optimally for each request to achieve the best possible latency. Compared to regular Sequence Paxos, this means choosing to use the fast path when it is likely successful, avoiding the message delay for forwarding the request. Compared to Fast Sequence Paxos, this means choosing the safer forwarding option when a fast-path attempt is considered risky, since a collision and its subsequent slow-path resolution and potential need for retrying introduce heavy latency penalties.

From the perspective of a follower, we can consider a fast path attempt successful only if its proposal achieves a fast quorum directly, without requiring a slow-path collision resolution. Conversely, a failure occurs whenever this optimal path is interrupted. A fast path attempt will fail if the target slot is claimed by a concurrent proposal by the leader through the regular path (majority quorum of `LEADERACCEPT` messages), claimed by a different follower's concurrent fast path proposal (fast quorum of `FASTACCEPT` messages), or stalled entirely by a collision that forces the cluster into a slow-path recovery.

To dynamically determine the appropriate operating mode, we design a heuristic that condenses real-time metrics into a score estimating the probability of a fast path succeeding, serving as the foundation for the mode predictor. This is done empirically via network simulations.

We repeatedly simulate consensus networks spanning a broad range of operating conditions. Each simulation instantiates a cluster consisting of between four and eight nodes. The aggregate proposal rate is sampled logarithmically between 5 and 50,000 proposals per second, while the baseline one-way network latency is sampled logarithmically between 0.2 ms and 200 ms. The proportion of proposals initiated by the leader is sampled independently, with the remaining proposal load distributed randomly among the followers to expose different mixtures of regular- and fast-path traffic.

To introduce heterogeneous network conditions, every communication link is assigned an independent latency based on the sampled baseline latency together with randomized per-link variation. Individual message transmissions additionally experience randomized latency jitter, allowing each

follower to observe a distinct network environment. Proposal generation and message delivery are simulated asynchronously using a discrete-event model.

The simulation duration is adjusted according to the aggregate proposal rate to ensure that each parameter configuration produces a sufficient number of proposal attempts while keeping the computational cost manageable. A total of 10,000 independent parameter configurations are simulated. After each simulation, every follower contributes one training sample consisting of locally observable runtime metrics together with the empirically measured fraction of proposals that completed successfully through the fast path. The parameter distributions are chosen to provide broad coverage of the operating space rather than to model a specific production workload. The resulting dataset is then used to investigate how different metrics influence fast-path success.

3.2.1 Important Metrics

To isolate the environmental variables that potentially impact fast-path outcomes, we define a set of system features tracked from the perspective of an individual follower. These metrics are divided into three categories driven by the structural trade-offs of the protocol: **Network Latency** captures time windows where proposals can be competed with; **Traffic and Proposal Rates** measure active, concurrent slot competition that could trigger a collision, and **System Scale** impacts the absolute quorum thresholds. The considered metrics are as follows:

Network Latency:

- *Latency to Leader*: Baseline message delay to the designated leader.
- *Latency to Majority Quorum*: The time required until a majority quorum can be reached with a message.
- *Latency to Fast Quorum*: The time required until a fast quorum can be reached with a message.
- *Maximum Latency*: Baseline message delay to the furthest node.

Traffic and Proposal Rates:

- *Leader Proposals/sec*: Volume of concurrent accept requests initiated by the leader. This is a measure about how many other requests take the regular path.

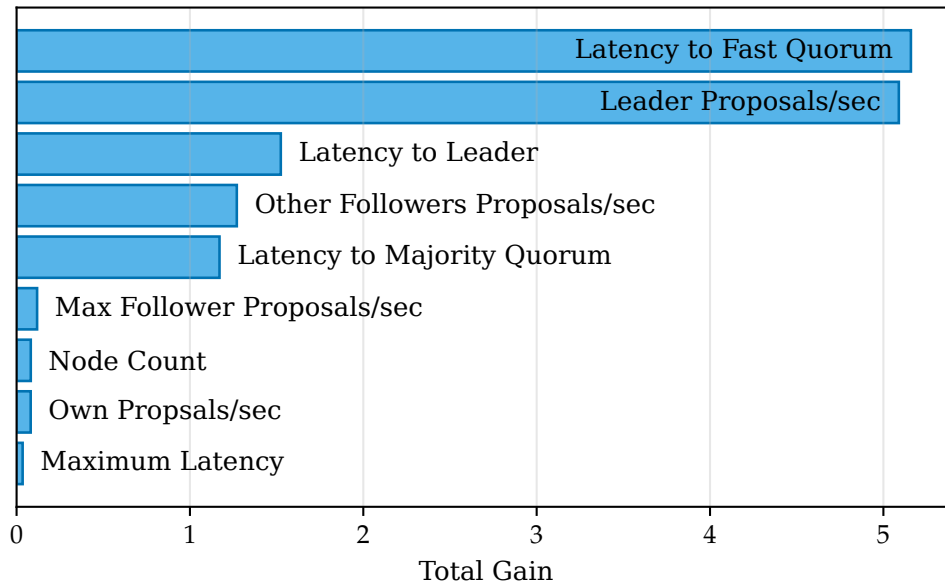


Figure 3.1: Feature importance for fast-path success measured by total information gain across the XGBoost model ensemble.

- *Own Proposals/sec*: The individual request rate of the follower.
- *Other Followers' Proposals/sec*: The aggregate concurrent request rate originating from all other followers. This is a measure about how many other requests take the fast path.
- *Max Follower Proposals/sec*: The peak request rate observed from a competing follower.

System Scale:

- *Node Count*: Total number of active nodes operating within the cluster.

To determine which of these metrics actually drive the system's behavior, we performed a feature importance analysis on the collected simulation data. An XGBoost regression model [47] was trained using the established features to predict the empirical fraction of proposals that completed successfully through the fast path for an individual follower. The dataset was randomly partitioned into training (70%), validation (15%), and test (15%) subsets. XGBoost models were trained on the training partition while monitoring the validation partition for early stopping to reduce overfitting. All reported evaluation metrics refer exclusively to the held-out test partition.

Using all the described metrics, the XGBoost model achieved a low **Root Mean Squared Error (RMSE)** of 0.00372 and a high *Coefficient of Determination* ($R^2 = 0.97458$). This R^2 score indicates that the selected metrics account for approximately 97.5% of the variance in fast-path success, validating that the input features successfully capture the underlying mechanics of the simulator.

The relative impact of each metric was quantified using the *Information Gain* of the underlying decision trees. This was selected because it measures the reduction in prediction error contributed by each feature over all tree splits in the ensemble. The result is illustrated in Figure 3.1.

XGBoost is deliberately employed as an exploratory analysis tool because it provides robust feature importance estimates while modeling highly nonlinear feature interactions. To minimize system overhead, however, the resulting score function should be simpler and efficient to compute, requiring as little runtime metric tracking as possible. For this purpose, the goal is to first reduce the feature space without losing much performance. From the original feature importance plot, we observe that latency to the fast quorum contributed the largest information gain. If we remove all other latency features and repeat the XGBoost training procedure, we achieve a **RMSE** of 0.00631 and R^2 score of 0.95689, which demonstrates that the model retains most of its predictive accuracy despite using a significantly smaller feature set. Figure 3.2 shows the resulting feature importance.

Now, keeping only the three most important features (*Latency to Fast Quorum*, *Leader Proposals/sec*, and *Other Followers' Proposals/sec*) increases the **RMSE** to 0.00798 and decreases the R^2 score to 0.94552, which is an acceptable trade-off because the model still explains over 94% of the variance while drastically simplifying runtime tracking. With these three established metrics, we now try to build a lightweight analytical approximation as an efficient score function for the final runtime predictor.

3.2.2 Score Function

To explore how the three remaining metrics interact, we can look at the time a proposal spends in flight. The *Latency to Fast Quorum* can be thought of as a window where a fast-path proposal is vulnerable to competition. During this time, requests from both the leader and other followers could potentially disrupt the fast path. Therefore, summing these two rates and multiplying by the latency gives a dimensionless value that could track the number of competing proposals issued during that window. This combination provides

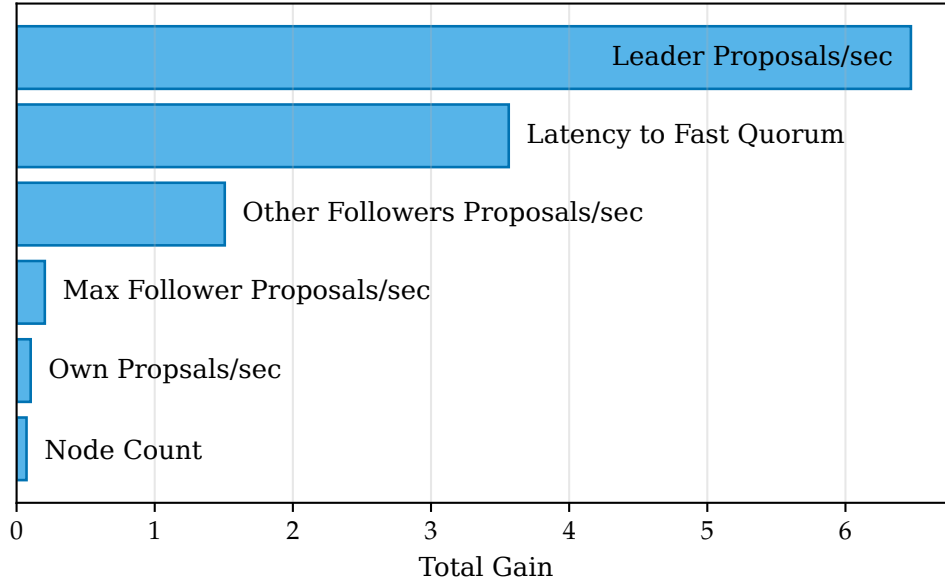


Figure 3.2: Feature importance for fast-path success with a reduced feature set measured by total information gain.

a plausible starting point for a well-defined score function, which is also underlined by the fact that an XGBoost model just on this metric produces a **RMSE** of 0.00806 an R^2 score of 0.94495. This is very close to the results achieved using the original features separately.

When visualizing the ”competing proposals” against the fast-path success rate in log-space (as done in Figure 3.3), a clear pattern emerges. The data exhibits a distinct sigmoid-like shape, with the success rate remaining near one for low values before dropping sharply and flattening out at zero.

To approximate this relationship, we fit a sigmoid function using the ”competing proposals”, x , which is the product of the fast quorum latency and the combined proposal rate. The model is formulated as follows:

$$\hat{\pi}(x) = \frac{1}{1 + e^{k(\ln(x) - \ln(x_0))}}$$

A curve fit yields a steepness factor of around $k \approx \frac{5}{4}$ and a parameter $x_0 \approx 0.5$, which marks the inflection point where the predicted success rate drops to 0.5. The latter is consistent with intuition, as when there is a 50% chance of a competing proposal, then it is imaginable that there is roughly a 50% chance of our fast-path attempt not being involved in a conflict.

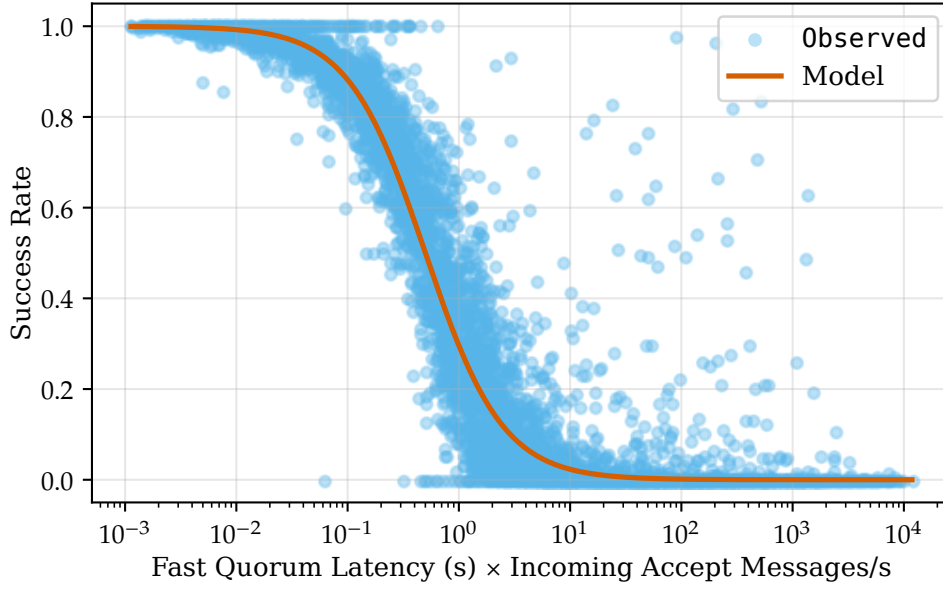


Figure 3.3: Observed fast-path success rate versus the expected number of competing proposals. The red line highlights the analytical heuristic score used for prediction.

Inserting these values and transforming the equation leads to the final heuristic model, as visualized in Figure 3.3, in the following form:

$$\vec{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \text{Fast Quorum Latency (s)} \\ \text{Incoming Accept Messages/s} \end{bmatrix}$$

$$\pi(\vec{x}) = \frac{1}{1 + (2 \cdot x_1 \cdot x_2)^{\frac{5}{4}}}$$

With this function, we achieve a **RMSE** of 0.00817 and an R^2 score of 0.94422 on the test data. This small trade-off in accuracy is acceptable to have a negligible computation and an interpretable score. A summary comparing the performance metrics of each model iteration against the original feature set is provided in Table 3.1. The function can now serve as the heuristic notion of uncertainty, as needed for **CRC**, around the mode prediction. An evaluation of its usability is presented in Section 4.2.

Model Configuration	RMSE	R^2 Score
XGBoost (Full Feature Set)	0.00372	0.97458
XGBoost (Reduced Latency Features)	0.00631	0.95689
XGBoost (Top Three Features)	0.00798	0.94552
XGBoost (Single Combined Metric)	0.00806	0.94495
Analytical Heuristic Function	0.00817	0.94422

Table 3.1: Performance comparison of feature selection and predictive models.

3.3 Risk-Controlled Mode Prediction

To bound the risk of failed fast-path attempts, the system incorporates the heuristic function into a risk-controlled prediction layer, outlined in this section. This framework wraps the scoring model to bound the empirical rate of fast-path failures relative to a user-specified risk level α in the long run. This failure rate is measured relative to the total number of append attempts initiated by the node, rather than only the subset of proposals where the fast-path was actively selected.

3.3.1 Risk Specification

To formulate the mode selection as a risk-control problem, the system’s outcomes and penalties must be strictly defined. We establish this formulation using three primary components: the label space, the loss function, and the user-defined risk level.

First, we define the operational label space of a proposal attempt as $\mathcal{Y} = \{\text{SUCCESS}, \text{NoSUCCESS}\}$. A **SUCCESS** indicates that a fast-path proposal from the perspective of the proposing follower achieves a fast quorum directly under the current network conditions. Conversely, a **NoSUCCESS** means that the attempt fails.

To bound the failure rate of fast paths, which is the risk we are trying to control, we have to penalize the false positives, where a fast-path is chosen and ends up being unsuccessful. If we fall back to the conservative/regular path whenever the **NoSUCCESS** label is included in the prediction set, the loss can be defined as such: The loss should be 1 whenever the true outcome is **NoSUCCESS**, but the prediction set fails to include **NoSUCCESS** (meaning the system confidently, but incorrectly, predicted a fast-path success) and 0

True Label	Prediction Set			
	$\emptyset$	$\{Success\}$	$\{NoSuccess\}$	$\{Success, NoSuccess\}$
Success	0	0	0	0
NoSuccess	1	1	0	0

Table 3.2: Loss based on true labels versus prediction sets.

otherwise. This loss is mapped using Table 3.2.

The design does not punish conservative predictions where the fast-path would have succeeded but the system safely defaulted to the regular path. The reason for this choice is that such a punishment simply does not fit into a loss function that has to satisfy the monotonicity properties relevant to provide the guarantees of CRC, as outlined in Section 2.2.3.

To mathematically evaluate this loss against our target failure rate, we must formally define the output of the prediction layer based on the heuristic established in Section 3.2. Let $\vec{x} \in \mathcal{X}$ denote the vector of real-time network features, and let $\pi(\vec{x})$ be the heuristic score representing the estimated probability of a fast-path success. We define a generalized scoring function $s(\vec{x}, y)$ for the label space $\mathcal{Y}$, where:

$$\begin{aligned} s(\vec{x}, \text{SUCCESS}) &= \pi(\vec{x}) \\ s(\vec{x}, \text{NoSUCCESS}) &= 1 - \pi(\vec{x}) \end{aligned}$$

We can then define the predictor as a set-valued function $C_\lambda : \vec{x} \rightarrow 2^{\mathcal{Y}}$ controlled by a calibration parameter $\lambda \in [0, 1]$. For a given proposal attempt, the prediction set is constructed by including any label whose score meets or exceeds the threshold $1 - \lambda$:

$$C_\lambda(\vec{x}) = \{y \in \mathcal{Y} \mid s(\vec{x}, y) \geq 1 - \lambda\}$$

This construction guarantees monotonicity with respect to λ . A maximally conservative parameter ($\lambda = 1$) lowers the inclusion threshold to 0, yielding the complete label space $C_1(X) = \{\text{SUCCESS}, \text{NoSUCCESS}\}$, which guarantees a safe fallback to the regular path. Conversely, as λ approaches 0, the threshold tightens, and the prediction set shrinks to only include labels with a high degree of heuristic certainty.

Finally, the framework bounds the risk using a user-specified risk level

$\alpha \in (0, 1)$, which dictates the target failure rate. If we had an exchangeable calibration set, the standard **CRC** procedure would provide the following guarantee for a new test point:

$$\mathbb{E}(\mathcal{L}(C_{\hat{\lambda}}(X_{test}), Y_{test})) \leq \alpha$$

Since real-life workloads are unlikely to be exchangeable, however, we instead use the rolling risk-control procedure (as defined in Section 2.2.4), which gives us the following for a stream of decision problems:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathcal{L}(C_{\lambda_t}(X_t), Y_t) \leq \alpha$$

Here, the converging value is bounded by α instead of being equal to it, as outlined previously. The reason is that in a setup where the fast-path always succeeds, the failure rate (and therefore the risk) is zero. Formally, our loss function does not satisfy the property that it has to produce a value higher than α when provided an empty prediction set since $\mathcal{L}(\emptyset, Success) = 0 < \alpha$.

3.3.2 Decision Layer

To execute runtime mode switching, each follower monitors local environment metrics. The fast quorum latency can be tracked continuously via a sliding window of recent message round-trip times to a fast quorum of peers, while the incoming proposal rate is calculated by recording the timestamps of arriving `Accept` messages from other nodes within a moving time window.

For every proposal attempt, the decision layer follows Algorithm 3.3. First, the tracked metrics are fed into the heuristic function to obtain probability scores for both outcomes in the label space $\mathcal{Y}$. Next, a prediction set is constructed by including any label whose assigned score satisfies the threshold condition bounded by the conformal parameter $\hat{\lambda}$. Intuitively, a threshold of $\hat{\lambda} = 1$ always returns the full label set, forcing a maximally conservative posture, whereas a threshold of $\hat{\lambda} = 0$ collapses the selection down to an empty set.

Finally, the contents of the prediction set are directly translated into an operational mode for the proposal. If the set contains the `NoSuccess` label, the node falls back to Regular Paxos, forwarding proposals to the leader. Conversely, if `NoSuccess` is absent from the prediction set, the node switches to FastPaxos mode and initiates fast-path broadcasts instead.

Algorithm 3.3 Conformal Mode Prediction

```

1: procedure GETPREDICTIONSET(features,  $\lambda$ )
2:   scores  $\leftarrow$  MODEL(features)
3:   predictionSet  $\leftarrow$  [ ]
4:   for each (label, score) in scores do
5:     if score  $\geq 1.0 - \lambda$  then
6:       Append label to predictionSet
7:     end if
8:   end for
9:   return predictionSet
10: end procedure

11: procedure GETNEWMODE(features)
12:   labels  $\leftarrow$  GETPREDICTIONSET(features,  $\hat{\lambda}$ )
13:   if NoSUCCESS  $\in$  labels then
14:     return REGULARPAXOS
15:   else
16:     return FASTPAXOS
17:   end if
18: end procedure

```

3.3.3 Calibration Strategy

To calibrate the prediction layer, nodes require ground-truth data indicating whether a fast-path attempt would have succeeded under the current network conditions. To gather this data without impacting safety on the active log, the system can employ a parallel data-gathering mechanism using shadow proposals.

Whenever a node opts for the regular path and forwards a proposal to the leader, it can simultaneously broadcast a shadow proposal tagged with a `TestAccept` status to all peers. Receiving followers treat `TestAccept` messages passively: if the target log slot is currently empty, the follower acknowledges the message by sending an `Accepted` response back to the original proposer, but it strictly does not write the entry to its local replicated log.

The original proposer accumulates these responses to determine the simulated outcome. If a `TestAccept` gathers a fast quorum of acknowledgments, the hypothetical fast-path attempt is deemed successful, and a `SUCCESS` label is recorded alongside the corresponding feature state. Conversely, if the actual log slot is committed by the leader via a `Decide` without the fast quorum

being formed, meaning many nodes have ignored this shadow proposal and the attempt is considered failed, it records a NoSUCCESS label. While this mechanism might occasionally underestimate fast-path viability, for instance, if the forwarded regular proposal is processed exceptionally fast or if an acknowledgment packet is dropped, it provides a sufficiently accurate proxy metric under stable network conditions.

Actual calibration can then happen two different ways: offline and online. In an offline calibration phase, the node collects a batch of shadow proposal outcomes to form a static dataset. The system then computes an optimal initial parameter, $\hat{\lambda}$ using the standard CRC approach, by employing a binary search algorithm to find the exact root where the empirical risk over the calibration set matches the significance level α . While this offline batch calibration does not provide formal theoretical guarantees for non-exchangeable future data, it can establish a sound baseline.

An online calibration phase utilizes the rolling risk control mechanism to incrementally update the threshold after each decision problem t . Using a defined learning rate γ , the parameter is adjusted based on the observed loss:

$$\hat{\lambda}_t \leftarrow \hat{\lambda}_t + \gamma \left(\mathcal{L}(C_{\hat{\lambda}_t}(X_t), Y_t) - \alpha \right)$$

This continuous adjustment bounds the failure rate and provides the long-term risk guarantees outlined previously.

Notably, the system does not need to issue shadow proposals during this online phase with our current loss function. Because of its definition, it evaluates to exactly zero whenever the prediction set contains the NoSUCCESS label, which forces the node to take the regular path. The true label of what *would* have happened on the fast-path becomes mathematically irrelevant for the parameter update. The loss can only be non-zero when the system chooses the fast-path and subsequently fails. As a result, the online risk-control loop can operate without the network overhead of shadow proposals.

Chapter 4

Analysis and Results

This chapter presents the empirical evaluation of the developed Conformal Consensus architecture. The primary goal is to determine whether risk-controlled mode prediction can capitalize on specific operating conditions where fast-path execution offers latency advantages while tracking target failure rates.

The adaptive system is evaluated against two static protocol baselines. The first baseline represents a Regular Paxos implementation using a trimmed-down version of the OmniPaxos [27] codebase, where all followers consistently forward proposals to the central leader via the conservative path. The second baseline is a static variant of the hybrid protocol in which followers unconditionally select the fast path for every proposal, which therefore closely resembles Fast Paxos.

After presenting the detailed experimental setup (see Section 4.1), the evaluation examines the system across four key performance and correctness dimensions. First, we assess whether the score function is a useful heuristic under dynamic request rates in Section 4.2. Second, Section 4.3 verifies whether the Rolling Risk Control layer empirically tracks the user-specified fast-path failure rates. Third, Section 4.4 analyzes end-to-end request latencies across multiple cluster topologies to identify whether adaptive mode switching can achieve performance gains. Finally, Section 4.5 concludes the chapter by quantifying the possible computational overhead that can be added by the introduced mechanisms.

4.1 Experimental Setup

The evaluation prototype is implemented in the Rust programming language, building directly upon a trimmed-down fork of the original OmniPaxos [27] codebase. To cleanly isolate the protocol variables associated with decentralized mode switching, several production-level features of OmniPaxos were omitted. The log is managed entirely via an in-memory storage layer, which enforces a fail-stop failure model where nodes do not recover from persistent disk state. Because the evaluation assumes a stable leader, a fully safe log synchronization phase at the conclusion of the prepare phase was omitted. Additionally, request batching is removed, ensuring that performance evaluation isolates fine-grained end-to-end latency rather than aggregate throughput.

Building upon this baseline, the consensus layer incorporates the protocol interoperability rules outlined previously. This includes adding the `AcceptStatus` field in both `Accept` and `Accepted` messages, implementing the followers' logic to broadcast proposals directly to peers via the fast path, and executing the tracking and deterministic collision-detection logic defined in Algorithm 3.1 and Algorithm 3.2 on the leader side. Follower-side update semantics are enforced to maintain safety: followers reject standard `FastAccept` and `LeaderAccept` messages if the target log index is already occupied, but permit `SlowAccept` commands from the leader to override existing entries during collision resolution.

When we operate in adaptive mode, the system implements dedicated tracking mechanisms for network features and success labels. To estimate the fast-quorum latency, the **Ballot Leader Election (BLE)** layer extends its standard heartbeat messages with a timestamp. Upon receiving a reply, the round-trip time is divided by two to compute individual one-way network latencies. Concurrently, the sequence Paxos layer tracks a running count of incoming `Accept` requests received within a sliding one-second window to estimate the proposal rate of other cluster nodes. Next to the shadow proposals, ground-truth labels are gathered by analyzing the final `AcceptStatus` of a node's own completed proposals once they are decided, where a resolved `FastAccept` signifies a fast-path success.

These features and labels are processed by the prediction layer implemented as an isolated module. It evaluates the model and executes the decision framework outlined in Algorithm 3.3. During runtime operations, the online calibration routine continuously updates the threshold using a fixed learning rate of $\gamma = 0.005$, adopting a baseline value from [48]. Before, an offline

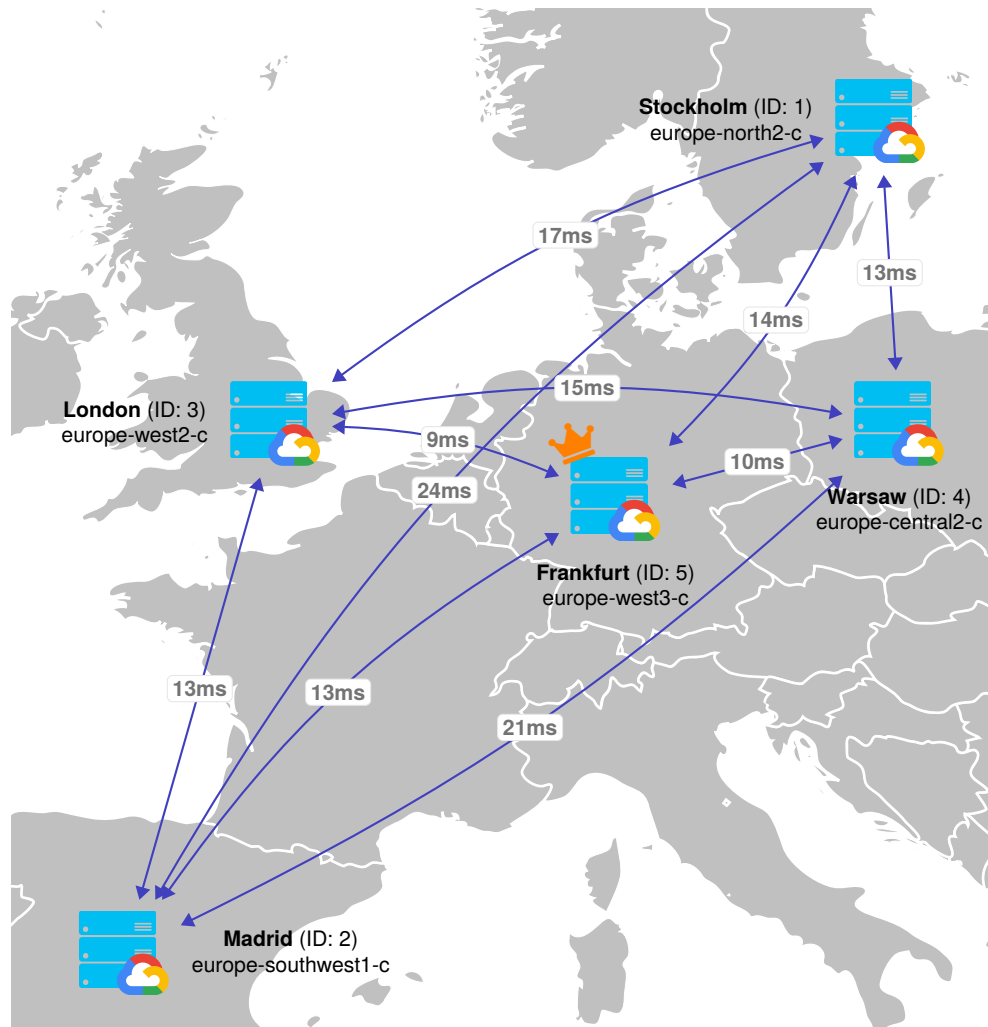


Figure 4.1: European cluster setup, including one-way latencies.

batch calibration phase is executed over a dataset collected over five minutes of traffic to bring the initial threshold directly into the appropriate operational range, from which point the small learning rate is assumed to be sufficient for ongoing updates.

4.1.1 State Machine Application

For the experiments, we used a linearizable, in-memory key-value store as the application-level replicated state machine. State transitions and queries are driven by three basic operations: inserting a key-value pair, deleting a

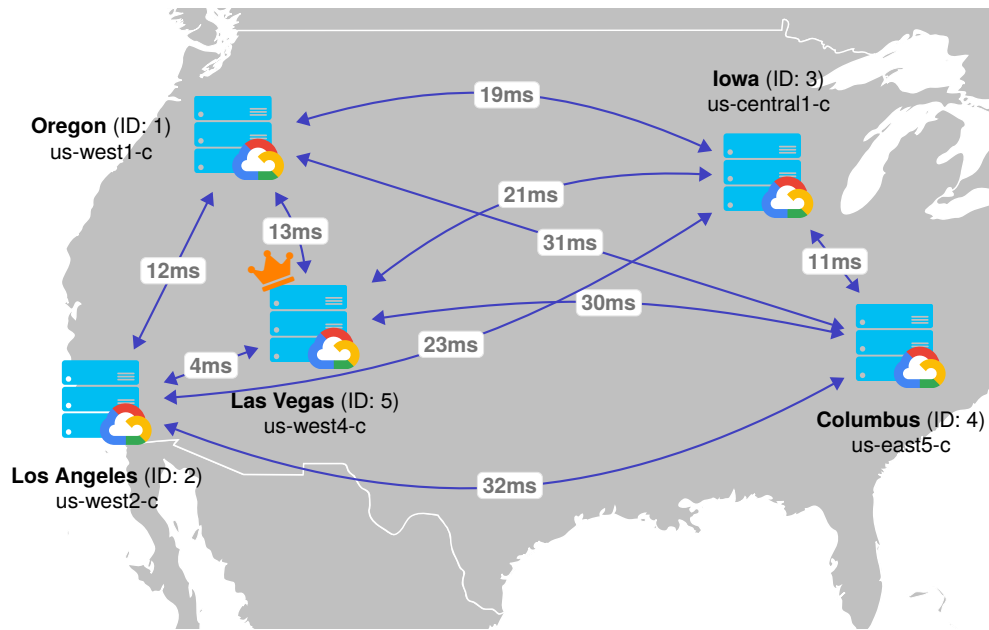


Figure 4.2: US cluster setup, including one-way latencies.

key, or retrieving a value based on a key. To guarantee strict linearizability and prevent stale data, both read and write operations are appended to the consensus log. As the consensus engine commits these entries, the server executes the commands sequentially once they are decided.

Client requests are encapsulated into a standardized command structure that includes a unique command identifier and a coordinator node identifier. Upon receiving a request, the coordinator node appends this command to the consensus log. After the log entry is committed and executed sequentially by the state machine, the node matching the recorded coordinator identifier handles routing the final execution outcome back to the originating client.

4.1.2 Node Setup

To evaluate the system under realistic network conditions, the prototype is deployed on **Google Cloud Platform (GCP)** homogeneous `e2-standard-4` instances, as detailed in Table 4.1. This instance class provides four virtual CPUs, guaranteeing at least one dedicated core per system component to prevent thread-scheduling bottlenecks. Architecturally, the system on each node is decoupled into three distinct Docker containers (`client`, `server`, and `daemon`) using host networking to bypass virtualization overhead.

Component	Configuration
Cloud Provider	Google Cloud Platform
Machine Type	e2-standard-4 (4 vCPUs, 16 GB RAM)
Operating System	Debian 13
Boot Disk Size	10 GB
Network Interface Type	Google Virtual NIC
Network Routing Tier	Premium
Provisioning Model	Standard
Container Environment	Docker (<code>-network host</code>)

Table 4.1: Hardware and software configuration specifications for the experimental cluster instances deployed on GCP.

The network daemon persists connections between nodes across different experiment runs, eliminating connection-establishment variance that would otherwise skew baseline latencies. Inter-process communication between the server and daemon is handled locally via Unix Domain Sockets.

The evaluation is conducted across two distinct 5-node clusters, located in Europe (Figure 4.1) and the US (Figure 4.2), to provide diverse latency profiles. Given a cluster size of $n = 5$, a majority quorum requires three nodes for the regular and slow paths, while the fast path requires a fast quorum of four nodes. Because this thesis represents an exploratory study of the protocol’s viability, the evaluation assumes a zero-failure environment. This isolates the adaptive mechanism under ideal operating conditions, leaving availability trade-offs for future work. To explore realistic routing conditions, the stable leader is deterministically assigned to the most geographically central node in each cluster: Frankfurt in Europe and Las Vegas in the US.

Tables 4.2 and 4.3 present the estimated baseline latencies from the perspective of a co-located client. This end-to-end measurement encapsulates the initial request, quorum gathering (fast or regular), potential collision resolution, the leader’s decide phase, local execution, and the final reply to the client. Co-locating the client simulates the architectural pattern of routing requests to the nearest edge replica while satisfying practical resource constraints.

The European topology exhibits a relatively distributed latency profile, where followers uniformly gain a slight advantage on a successful fast path because broadcasting to a localized fast quorum is faster than forwarding.

Node	Regular Path	Fast Path	Slow Path
Stockholm	48ms	40ms	60ms
Madrid	46ms	44ms	64ms
London	38ms	35ms	55ms
Warsaw	40ms	37ms	57ms
Frankfurt	20ms	26ms	46ms

Table 4.2: Estimated end-to-end latencies in the European cluster setup using regular, fast, or slow execution paths for an entry appended by a co-located client.

Node	Regular Path	Fast Path	Slow Path
Oregon	52ms	53ms	79ms
Los Angeles	21ms	48ms	74ms
Iowa	68ms	53ms	79ms
Columbus	86ms	66ms	92ms
Las Vegas	26ms	42ms	68ms

Table 4.3: Estimated end-to-end latencies in the US cluster setup using regular, fast, or slow execution paths for an entry appended by a co-located client.

Conversely, the US topology has a more polarized profile. While distant nodes benefit significantly from the fast path, nodes physically close to the leader, such as Oregon and Los Angeles, incur a latency penalty when attempting to gather four distributed votes compared to simply forwarding their request. Since these nodes have zero mathematical incentive to switch operating modes, their local modes are locked to Regular Paxos in the adaptive experiments.

4.1.3 Workload Pattern

The experiments use a highly bursty, low-contention workload, ensuring periods where the fast path is highly viable are interspersed with periods of high collision risk, making the optimal strategy fluctuate between Fast Paxos and Regular Paxos. Traffic generation is driven by local events at the clients. The baseline traffic between events is set to zero **Requests per**

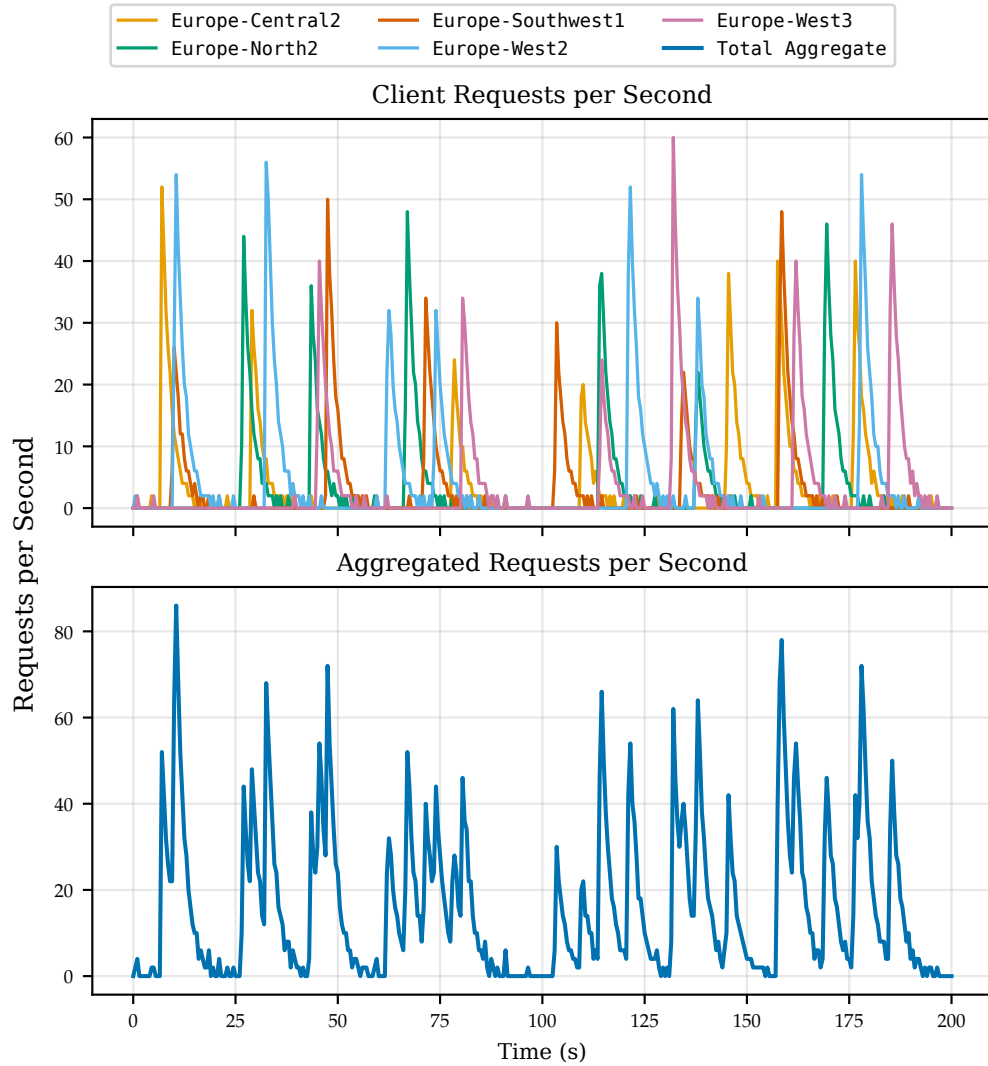


Figure 4.3: Sample of the client workload pattern used for experimentation.

Second (RPS). When a local event triggers at a node, the client injects a rapid burst of around 50 peak **RPS**. This simulates a sudden influx of client activity that normalizes along an exponential decay curve with a rate of 0.5. Burst timing is unpredictable: intervals between events are sampled from a normal distribution with a 30-second mean and a 15-second standard deviation. A jitter factor of 0.5 applies randomized variance to the peak height of each burst.

Each geographic server is driven by a single, independent client process sending requests. This setup does not impact collision rates, as collisions inherently occur across the network between different servers rather than locally. Because each of the nodes generates localized bursts independently,

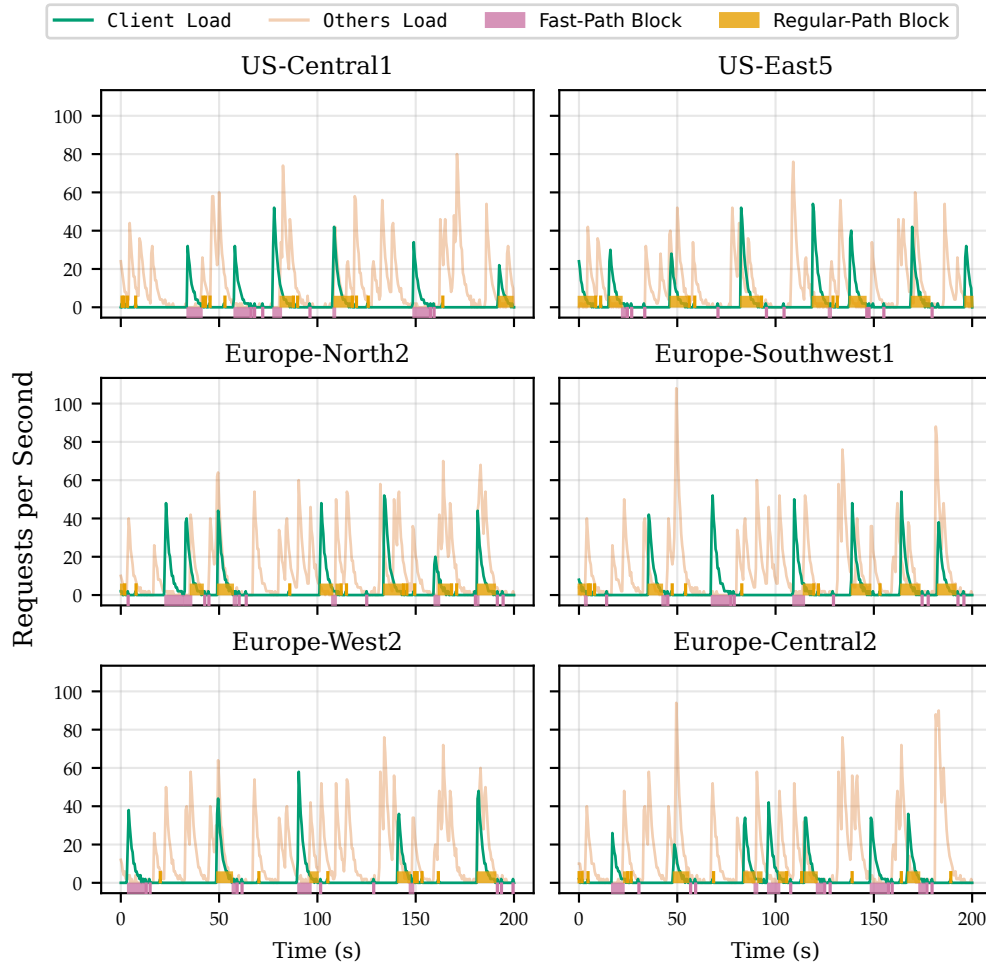


Figure 4.4: Heuristic score path selection of different nodes depending on their own load and other servers' load under a static threshold of 0.1.

the aggregate system load dynamically fluctuates. Figure 4.3 depicts a sample of this workload pattern, showing both the request trace for individual servers and the overall aggregate load. All evaluation runs are executed for a total duration of ten minutes, preceded by a one-minute warm-up period and potential batch calibration routines.

Under sustained, heavy traffic, standard replicated log protocols like OmniPaxos [27] or Raft [23] are likely strictly superior due to high collision rates on the fast path. If Conformal Consensus can show latency benefits under the ideal workload in a realistic cluster setup (as employed here), further studies can be made on practicality.

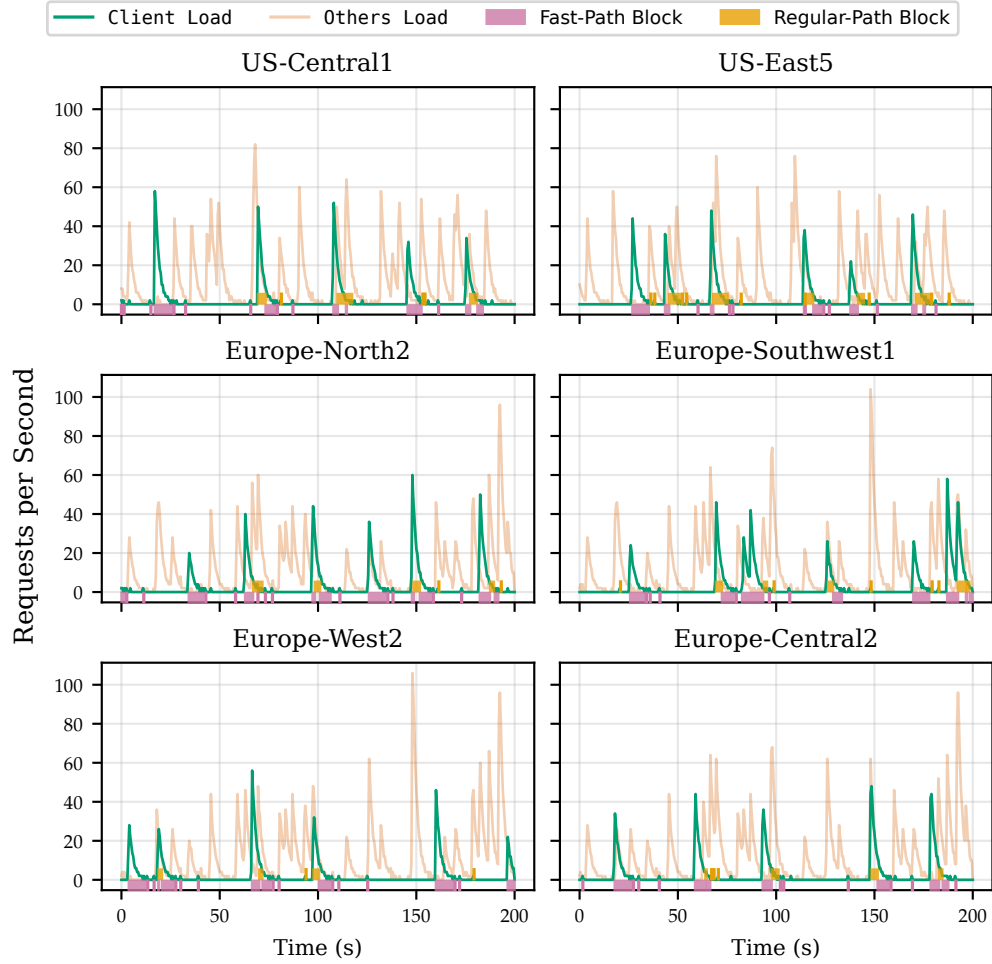


Figure 4.5: Heuristic score path selection of different nodes depending on their own load and other servers' load under a static threshold of 0.5.

4.2 Score Function Efficacy

The objective of this experiment is to demonstrate that the heuristic function derived in Section 3.2 acts as a responsive signal for fast-path viability under dynamic workload conditions, establishing a practical basis for risk control. To isolate the heuristic, the system operates using a static decision threshold. A node attempts a fast-path broadcast if and only if the calculated score $\pi(\vec{x})$ strictly exceeds $1 - \alpha$, where α represents the target risk level. We evaluate this using two static settings: $\alpha = 0.1$, requiring a 90% estimated success probability to choose the fast path, and $\alpha = 0.5$, requiring a 50% estimated success probability. To simplify data collection and presentation, leader-side

retrying of failed proposals is disabled.

Figures 4.4 and 4.5 illustrate the operating mode transitions against active network traffic over a 200-second window for adaptive nodes. Under the conservative threshold ($\alpha = 0.1$), the fast path is selected only when the load of other nodes is very low while reacting quickly to traffic changes. This minimizes collisions, indicating that the score function can successfully identify optimal moments to initiate fast paths.

When the required success rate is decreased to 50%, nodes attempt the fast path significantly more often, falling back to the regular path only during substantial load spikes from other nodes. This behavioral shift demonstrates that the score function captures a continuous probability gradient rather than acting as a simple binary switch.

Analysis of the raw data further validates this gradient. The local fast-path error rate ranges between 0.5% and 2% for the more conservative configuration ($\alpha = 0.1$), compared to between 5% and 13% under the relaxed setting ($\alpha = 0.5$). This confirms that adjusting the decision threshold modulates the actual risk in the expected manner, validating that the risk control mechanisms are built on a solid predictive foundation.

4.3 Empirical Validation of Risk Control

Figure 4.6 evaluates the effectiveness of Conformal Consensus in tracking the fast-path failure rate across multiple risk-level settings. This tracking behavior is noticeably successful and consistent across the evaluated nodes in both the European and US clusters, indicating that the online calibration routine functions as intended.

In certain cases, the actual error rate falls significantly below the target threshold. For instance, at the London node (*europa-west2*), the actual error rate plateaus and remains strictly below the target limit for risk levels of 0.4 and 0.5. This occurs because the fast-path utilization at these nodes has already reached 100%. Once the system exclusively selects the fast path, the failure rate is determined entirely by the baseline network collision rate and cannot mathematically increase further, regardless of higher target risk allowances.

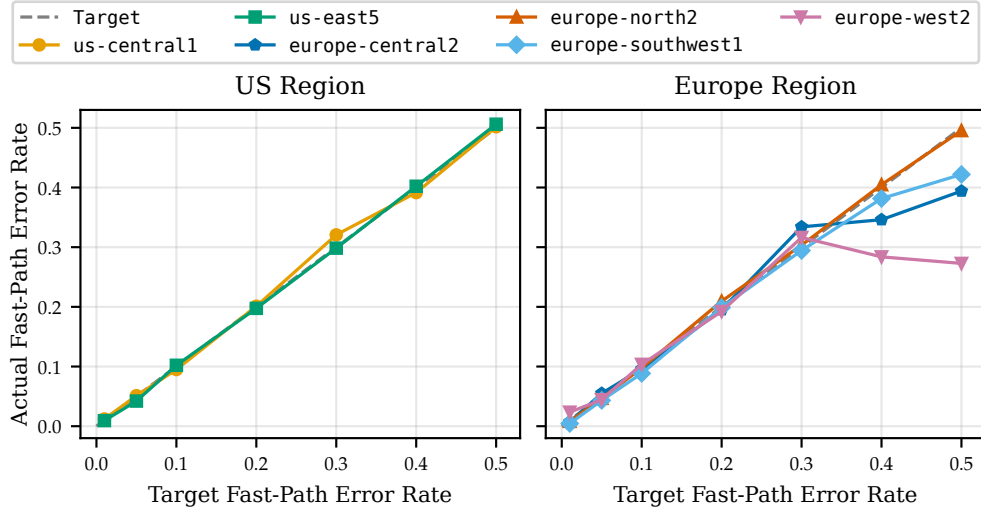


Figure 4.6: Comparison of the target fast-path error rate and the actual observed fast-path error rate across nodes in the US and European clusters.

4.4 Latency Study

We analyze the end-to-end request latencies of Conformal Consensus under different risk levels compared to static Regular Paxos and Fast Paxos baselines. Figures 4.7 and 4.8 display the mean, median, and tail latencies (P10, P90, P95, P99) for both regions under the bursty workload pattern described in Section 4.1.

In the European cluster experiment, Regular Paxos maintains a flat latency profile across all percentiles (P10–P99), demonstrating strong resilience to workload contention. In contrast, statically choosing fast-path achieves lower P10 latency for followers but produces the widest overall latency spread, suffering substantial latency inflation at higher percentiles (P90, P95, and P99). This can also be seen at the leader node (*europe-west3*). Because high-percentile penalties heavily impact the average, static Fast Paxos reduces mean latency only when the geographic fast-path advantage is substantial, as seen at the Stockholm node (*europe-north2*). Where fast-path benefits are marginal, its mean latency regresses relative to Regular Paxos.

Conformal Consensus dampens the extreme tail latencies. At a conservative risk level ($\alpha = 0.01$), for instance, latency inflation is restricted strictly to the P99 tail and remains unobservable at P90 and P95. Simultaneously, the adaptive mechanism preserves the low P10 latency of Fast Paxos, successfully capturing and increasing mean latency improvements on

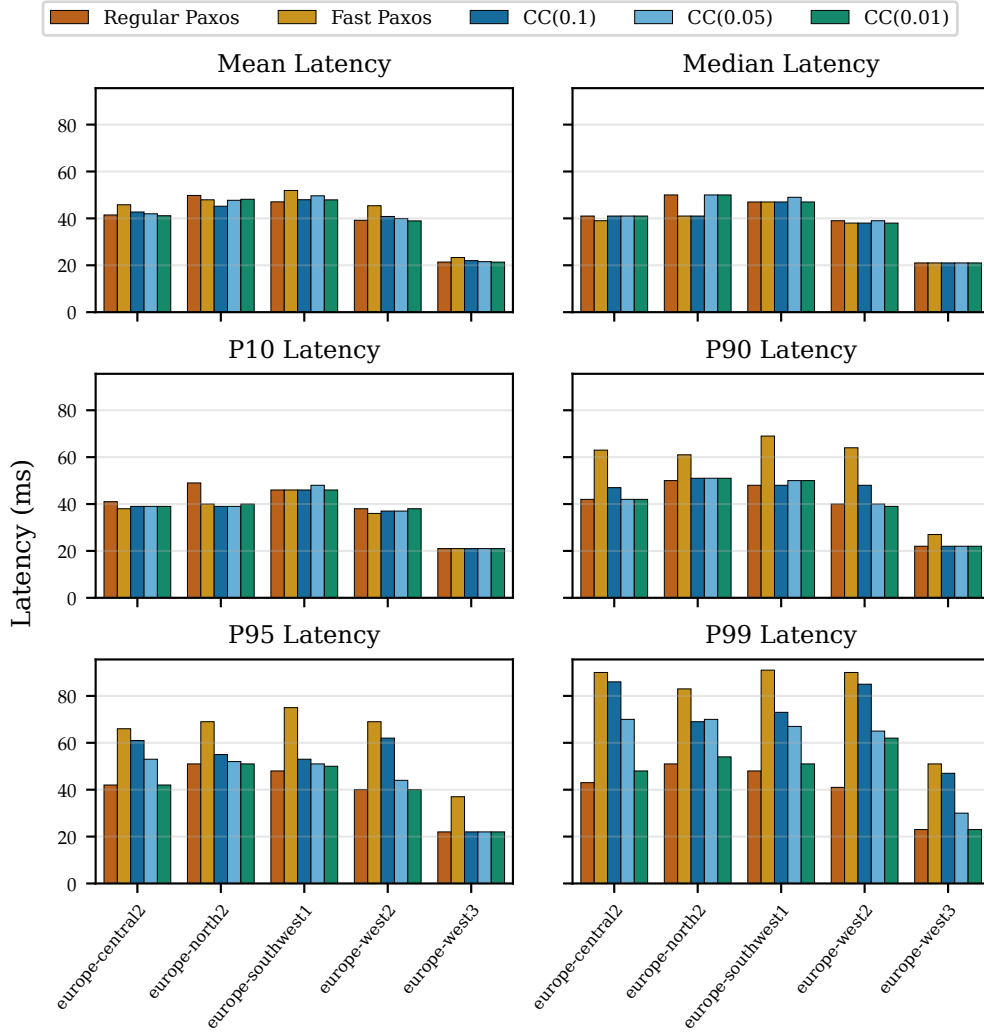


Figure 4.7: End-to-end latency comparison for the European cluster setup between Regular Paxos, Fast Paxos, and Conformal Consensus with various risk levels.

nodes with large fast-path advantages. However, where fast-path gains are minimal, residual slow-path fallbacks at high percentiles still cause a slight regression in mean latency compared to pure Regular Paxos.

Finally, it can be observed that the target risk level α establishes a direct trade-off between stability and latency reduction. Lowering α systematically increases tail dampening and tightens the overall latency spread toward Regular Paxos, though this added stability proportionally diminishes potential mean latency gains.

To mitigate the unpredictable cloud network variance observed in the

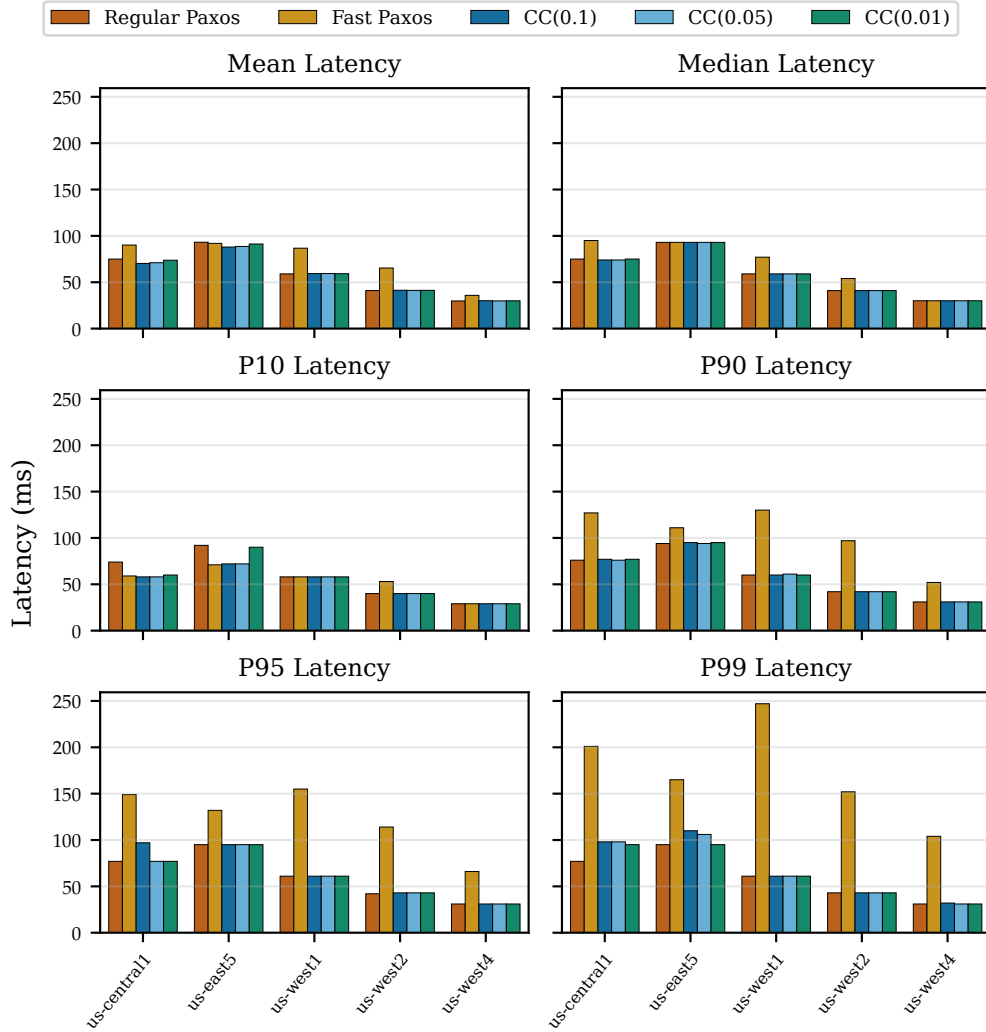


Figure 4.8: End-to-end latency comparison for the (simulated) US cluster setup between Regular Paxos, Fast Paxos, and Conformal Consensus with various risk levels.

US cluster setup, all nodes were deployed within a single geographic zone (`us-west4-c`) for the latency experiment, with the regional latency profile injected on the application side. Under these conditions, the Regular and Fast Paxos baselines exhibit the same general performance trends observed in the European cluster.

As discussed in Section 4.1.2, the Conformal Consensus configuration is set up to continuously route `us-west1`, `us-west2`, and the leader (`us-west4`) through the regular path. The latencies for these nodes perfectly mirror the Regular Paxos baseline across all percentiles through the added

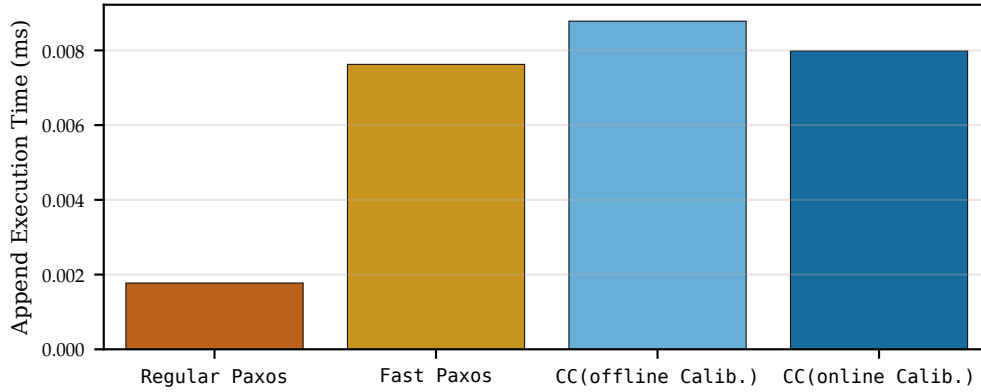


Figure 4.9: Average execution times of an append attempt by followers across operating modes.

stability. For the actively adapting nodes, higher theoretical fast-path gains result in a noticeable reduction in mean latency, making Conformal Consensus slightly outperform both static baselines in this specific setup.

4.5 Overhead Analysis

Conformal Consensus can introduce overhead in two primary areas: per-request mode evaluation during follower append attempts and threshold computation during batch calibration.

Figure 4.9 shows the average local execution time per append attempt on a follower across operating modes. On average, Regular Paxos requires less than 0.002 ms for this function execution because the follower only forwards incoming requests to the leader. Fast Paxos increases this due to proposal broadcasting to peers. Adding CRC, which includes evaluating the heuristic score, generating the prediction set, and sending potential shadow proposals during offline calibration, incurs less than 0.01 ms of combined compute overhead per request. Given that end-to-end network latencies in both cluster topologies are on the order of tens of milliseconds (Section 4.1), this local compute overhead is negligible and has no practical impact on request latency.

The second potential source of overhead is batch calibration, which runs a binary root-finding algorithm to establish an initial threshold $\hat{\lambda}$. As shown in Figure 4.10, execution time scales linearly with dataset size. For a dataset of 3,000 samples (collected over 15 minutes under the described workload pattern), the algorithm computes the threshold in approximately 15 ms. Batch calibration is entirely optional and serves primarily to establish a

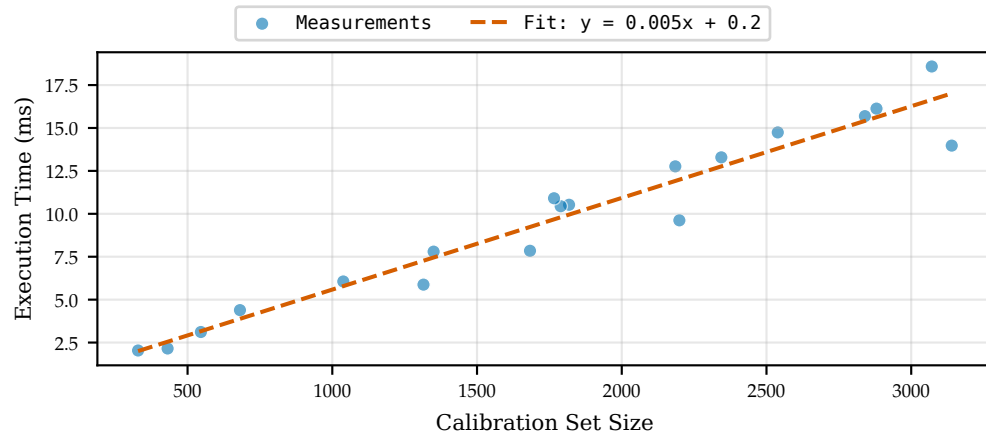


Figure 4.10: Threshold computation time during batch calibration relative to calibration set size.

baseline threshold prior to running online calibration. Because data collection can occur during regular operations, the root-finding routine can execute asynchronously, introducing no runtime disruption to the main consensus path with such a low computational load.

Chapter 5

Discussion

This chapter attempts to synthesize the findings from the analysis to assess the broader implications of the results and the practicability of the proposed design. We first evaluate the effectiveness of risk control in providing formal behavioral bounds in Section 5.1, followed by an analysis of the fundamental constraints that limit fast-path execution in Section 5.2. Building on these observations, Section 5.3 assesses the practical viability of the conformal consensus approach compared to established architectures. Finally, we detail the theoretical and methodological limitations of the current prototype in Section 5.4 and outline essential directions for future work in Section 5.5.

5.1 The Power of Formal Guarantees

The empirical evaluation confirms that Rolling Risk Control can effectively regulate fast-path routing with sub-millisecond overhead. By dynamically tracking the target risk and error rates across varying geographic clusters, the online calibration routine demonstrates that statistical methods can reliably govern adaptive protocol behavior without becoming a computational bottleneck. Specifically, the analysis confirms that the target risk parameter can provide a simple, effective interface for managing the trade-off between optimistic execution and system stability. Because the conformal wrapper provides a risk guarantee regardless of the underlying model’s accuracy, developers are free to employ anything from simple, locally observable heuristics to complex machine learning models.

While this thesis evaluates CRC specifically in the context of consensus fast paths, the underlying methodology is applicable to a much broader class of distributed algorithms. Beyond this work’s attempt at consensus routing,

online risk calibration could be applied to several areas of distributed systems that rely on uncertain runtime decisions. Some consensus-relevant examples could include message batching, leader migration or cluster reconfiguration, quorum selection, or speculative execution on the application side.

Overall, the empirical results demonstrate that conformal methods, specifically Rolling Risk Control, can effectively regulate distributed routing decisions when wrapped around lightweight heuristics. This work offers a template for integrating formal guarantees into distributed protocols, showing that they are highly accessible, simple to integrate, and therefore practical for regulating distributed systems.

5.2 Fast Path Viability

The theoretical latency reduction offered by the fast path was shown to be modest from the outset. In practice, the requirement for a larger fast quorum counteracts much of the theoretical benefit, particularly when the leader is strategically positioned relative to the rest of the cluster. This observation aligns with existing literature questioning the practical end-to-end performance gains of fast-path consensus over classic variants [18].

Because every proposal directly competes for the next sequential index in the log, contention is also an inherent property of the total-order abstraction. This structural bottleneck is the motivation behind partially ordered consensus designs, which exploit application-side command commutativity to avoid strict sequential conflict [15]. The derived heuristic already highlights that the operational frame in which the fast path succeeds reliably is remarkably narrow for replicated logs. According to it, under one-way network latencies of approximately 20 ms, which is common for an intra-continental cluster, sustaining a 90% fast-path success probability restricts the aggregate request rate across all other nodes to merely four requests per second.

Additionally, there exists a fundamental asymmetry in the cost-benefit trade-off: a successful fast-path proposal saves at most one message delay, whereas a collision triggers collision detection, slow-path recovery, and potential retries. Wide-area deployments amplify this imbalance. While geo-replicated systems stand to gain the most from eliminating forwarding delays, their higher network latencies proportionally expand the time window during which concurrent proposals overlap. Consequently, the environments where optimistic fast paths are theoretically most desirable are precisely those where their underlying assumptions are weakest.

Ultimately, the optimism of fast-path execution for replicated logs is

fragile, delivering negligible end-to-end latency gains even under ideal conditions, as shown in the analysis.

5.3 Practicality of Conformal Consensus

Modern, replicated log consensus protocols like OmniPaxos [27] or Raft [23] offer a highly dependable, extensible, and failure-tolerant foundation that is strictly superior under sustained, heavy traffic. Furthermore, significant performance improvements can already be extracted from these classic architectures through system-level optimizations, such as flexible phase quorums [40], caching [49], and strategic leader placement and general cluster setup optimization.

The analysis demonstrates, however, that slight end-to-end latency gains of a hybrid protocol materialize under highly idealized conditions. Practical viability strictly demands a highly stable cluster topology with minimal node failures, workloads characterized by periods of low contention where fast paths can succeed, and specific geographic configurations where fast-path routing holds a distinct physical advantage. Additionally, deployment can only be imagined when the goal is to decrease end-to-end latency and the across-node message passing is the primary bottleneck. This casts severe doubt as to the practicality of the hybrid protocol presented. If a complete hybrid protocol can be designed, there is still a high chance that it generally matches or outperforms Fast Paxos throughout.

Whether employing the CRC/Rolling Risk Control layer in the way it was designed aids practicality cannot be fully determined based on the analysis, but it at least provides a mathematically grounded, interpretable tuning knob with the fast-path error rate.

5.4 Limitations

First and foremost, the proposed hybrid consensus protocol is strictly exploratory and incomplete. It currently lacks formal safety and liveness proofs, and it remains fundamentally unverified whether a fully safe and lively specification of this design is theoretically achievable. Methodologically, the predictor evaluated whether a fast-path execution can succeed, rather than determining if it will be faster than the standard execution path. Under conditions of low contention, network latency variations may allow a fast path to succeed without delivering any net performance benefit over the standard

consensus flow. Formal guarantees based on end-to-end latency likely provide more insight, although it remains unclear if this is implementable.

Furthermore, several implementation decisions and environmental constraints affected the depth of the analysis. The adaptive mechanism relied on a fixed learning rate of 0.005, adopted from [48]. Because no dedicated hyperparameter study was conducted, the system relied on batch calibration to shift the threshold into an operational range from where a small learning rate was assumed to be sufficient. This leaves the true sensitivity of the learning rate unexplored. From a system perspective, the prototype operates entirely in memory and omits the disk I/O required for durable logging and recovery. While disk overhead is likely not the dominant latency factor in the distributed setups studied, the computational demands of the adaptive variant could introduce unmeasured storage bottlenecks. Finally, unpredictable network jitter in the public cloud environment, introducing notable latency variance within and across test runs, as well as limited resource availability, severely hindered progress in the evaluation.

5.5 Future Work

The primary theoretical priority is to fully formalize the hybrid consensus protocol. This involves developing rigorous safety and liveness proofs, specifying explicit requirements for safe leader election, and defining a formal log synchronization phase during leader transitions. From a methodological standpoint, future evaluations could investigate transitioning from predicting fast-path success to providing concrete guarantees on total end-to-end latency. Ensuring that the fast path is triggered only when it yields a real latency reduction would eliminate the need for extensive prior network topology mapping. The success heuristic could still be a valuable part in achieving this.

From a practical infrastructure standpoint, future implementations should incorporate durable logging to stable storage. This will allow researchers to evaluate how the hybrid mechanism performs under realistic I/O constraints. Finally, integrating a similar adaptive risk-controlled framework into established consensus architectures, such as Nezha [17], represents a promising direction. Combining adaptive prediction with a protocol already optimized for fast-path execution could directly address the primary applicability bottlenecks observed in this study.

Chapter 6

Conclusions

This thesis investigated whether adaptive regulation of the fast path devised for Fast Paxos can improve the performance of replicated logs while retaining the abstraction of a totally ordered log. To explore this idea, this thesis presented Conformal Consensus, which combines an exploratory hybrid consensus protocol with decentralized mode switching, a lightweight heuristic for estimating fast-path success, and an online conformal mechanism for regulating fast-path decisions according to a target error rate.

The conducted experiments allow answering the posed research questions outlined in Section 1.1. First, the results indicate that Rolling Risk Control can successfully track the fast-path error rate around a simple success estimation heuristic in the explored hybrid protocol. Despite relying only on locally observable metrics, the adaptive regulator was able to maintain the desired risk level over time within the evaluated distributed environments.

Second, the evaluation shows that adaptive path selection can slightly reduce mean end-to-end latency compared to both Regular Sequence Paxos and the implemented Fast Paxos variant. These improvements, however, were observed primarily under stable and highly favorable operating conditions, including a deliberately engineered workload with bursty, low-contention phases that created opportunities for successful fast-path execution. Consequently, the results should be interpreted as evidence that fast-path regulation can be beneficial under highly suitable conditions.

Overall, this work suggests that conformal methods can be applied beyond traditional prediction tasks to regulate protocol-level decisions in distributed systems. While the practical viability of the Conformal Consensus in production environments is at least questionable due to limited applicability of the traditional fast path, the presented design and evaluation indicate that

combining statistical guarantees of conformal methods with distributed data management (and specifically consensus algorithms) is a feasible research direction that merits further study.

References

- [1] Coulouris, J. Dollimore, and T. Kindberg, *Distributed Systems: Concepts and Design (4th Edition) (International Computer Science)*. USA: Addison-Wesley Longman Publishing Co., Inc., 2005. ISBN 0321263545. [Online]. Available: <https://dl.acm.org/doi/10.5555/1202138>
- [2] M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of distributed consensus with one faulty process,” *J. ACM*, vol. 32, no. 2, p. 374–382, Apr. 1985. doi: 10.1145/3149.214121. [Online]. Available: <https://doi.org/10.1145/3149.214121>
- [3] C. Dwork, N. Lynch, and L. Stockmeyer, “Consensus in the presence of partial synchrony,” *J. ACM*, vol. 35, no. 2, p. 288–323, Apr. 1988. doi: 10.1145/42282.42283. [Online]. Available: <https://doi.org/10.1145/42282.42283>
- [4] L. Lamport, “Paxos made simple,” *ACM SIGACT News (Distributed Computing Column)* 32, 4 (Whole Number 121, December 2001), pp. 51–58, December 2001. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/paxos-made-simple/>
- [5] ———, “Fast paxos,” *Distributed Computing*, vol. 19, no. 2, pp. 79–103, October 2006. doi: 10.1007/s00446-006-0005-x. [Online]. Available: <https://doi.org/10.1007/s00446-006-0005-x>
- [6] W. Zhao, “Fast paxos made easy: Theory and implementation,” *International Journal of Distributed Systems and Technologies*, vol. 6, no. 1, p. 15–33, Jan. 2015. doi: 10.4018/ijdst.2015010102. [Online]. Available: <http://doi.org/10.4018/ijdst.2015010102>
- [7] F. B. Schneider, “Implementing fault-tolerant services using the state machine approach: a tutorial,” *ACM Comput. Surv.*, vol. 22, no. 4, p.

- 299–319, Dec. 1990. doi: 10.1145/98163.98167. [Online]. Available: <https://doi.org/10.1145/98163.98167>
- [8] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, and D. Woodford, “Spanner: Google’s globally distributed database,” *ACM Trans. Comput. Syst.*, vol. 31, no. 3, Aug. 2013. doi: 10.1145/2491245. [Online]. Available: <https://doi.org/10.1145/2491245>
- [9] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, and P. Mattis, “Cockroachdb: The resilient geo-distributed sql database,” in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’20. New York, NY, USA: Association for Computing Machinery, 2020. doi: 10.1145/3318464.3386134. ISBN 9781450367356 p. 1493–1509. [Online]. Available: <https://doi.org/10.1145/3318464.3386134>
- [10] “etcd: A distributed, reliable key-value store,” <https://etcd.io>, accessed: 2026-01-13.
- [11] M. Burrows, “The chubby lock service for Loosely-Coupled distributed systems,” in *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06)*. Seattle, WA: USENIX Association, Nov. 2006. [Online]. Available: <https://www.usenix.org/conference/osdi-06/chubby-lock-service-loosely-coupled-distributed-systems>
- [12] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, “ZooKeeper: Wait-free coordination for internet-scale systems,” in *2010 USENIX Annual Technical Conference (USENIX ATC 10)*. USENIX Association, Jun. 2010. [Online]. Available: <https://www.usenix.org/conference/usenix-atc-10/zookeeper-wait-free-coordination-internet-scale-systems>
- [13] C. Stathakopoulou, M. Pavlovic, and M. Vukolić, “State machine replication scalability made simple,” in *Proceedings of the Seventeenth European Conference on Computer Systems*, ser. EuroSys ’22. New York, NY, USA: Association for Computing Machinery, 2022. doi: 10.1145/3492321.3519579. ISBN 9781450391627 p. 17–33. [Online]. Available: <https://doi.org/10.1145/3492321.3519579>

- [14] F. Ryabinin, A. Gotsman, and P. Sutra, “SwiftPaxos: Fast Geo-Replicated state machines,” in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. Santa Clara, CA: USENIX Association, Apr. 2024. ISBN 978-1-939133-39-7 pp. 345–369. [Online]. Available: <https://www.usenix.org/conference/nsdi24/presentation/ryabinin>
- [15] I. Moraru, D. G. Andersen, and M. Kaminsky, “There is more consensus in egalitarian parliaments,” in *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, ser. SOSP ’13. New York, NY, USA: Association for Computing Machinery, 2013. doi: 10.1145/2517349.2517350. ISBN 9781450323888 p. 358–372. [Online]. Available: <https://doi.org/10.1145/2517349.2517350>
- [16] S. J. Park and J. Ousterhout, “Exploiting commutativity for practical fast replication,” in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. Boston, MA: USENIX Association, Feb. 2019. ISBN 978-1-931971-49-2 pp. 47–64. [Online]. Available: <https://www.usenix.org/conference/nsdi19/presentation/park>
- [17] J. Geng, A. Sivaraman, B. Prabhakar, and M. Rosenblum, “Nezha: Deployable and high-performance consensus using synchronized clocks,” *Proc. VLDB Endow.*, vol. 16, no. 4, p. 629–642, Dec. 2022. doi: 10.14778/3574245.3574250. [Online]. Available: <https://doi.org/10.14778/3574245.3574250>
- [18] F. Junqueira, Y. Mao, and K. Marzullo, “Classic paxos vs. fast paxos: caveat emptor,” in *Proceedings of the 3rd Workshop on on Hot Topics in System Dependability*, ser. HotDep’07. USA: USENIX Association, 2007, p. 18–es. [Online]. Available: <https://dl.acm.org/doi/10.5555/1323140.1323158>
- [19] G. Shafer and V. Vovk, “A tutorial on conformal prediction,” *J. Mach. Learn. Res.*, vol. 9, p. 371–421, Jun. 2008. [Online]. Available: <https://dl.acm.org/doi/abs/10.5555/1390681.1390693>
- [20] A. N. Angelopoulos and S. Bates, “Conformal prediction: A gentle introduction,” *Foundations and Trends® in Machine Learning*, vol. 16, no. 4, pp. 494–591, 2023. doi: 10.1561/22000000101. [Online]. Available: <http://doi.org/10.1561/22000000101>

- [21] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, and T. Schuster, “Conformal risk control,” 2025. [Online]. Available: <https://arxiv.org/abs/2208.02814>
- [22] A. Håkansson, “Portal of research methods and methodologies for research projects and degree projects,” in *Proceedings of the International Conference on Frontiers in Education: Computer Science and Computer Engineering (FECS'13)*. CSREA Press U.S.A., 2013, pp. 67–73. [Online]. Available: <https://worldcomp-proceedings.com/proc/p2013/FEC2979.pdf>
- [23] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*. Philadelphia, PA: USENIX Association, Jun. 2014. ISBN 978-1-931971-10-2 pp. 305–319. [Online]. Available: <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [24] C. Cachin, R. Guerraoui, and L. Rodrigues, *Introduction to Reliable and Secure Distributed Programming*, 2nd ed. Berlin, Heidelberg: Springer, 2011. ISBN 978-3-642-15260-3. [Online]. Available: <https://doi.org/10.1007/978-3-642-15260-3>
- [25] M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of distributed consensus with one faulty process,” *J. ACM*, vol. 32, no. 2, p. 374–382, Apr. 1985. doi: 10.1145/3149.214121. [Online]. Available: <https://doi.org/10.1145/3149.214121>
- [26] S. Haridi, L. Kroll, and P. Carbone, “Lecture notes on leader-based sequence paxos - an understandable sequence consensus algorithm,” *CoRR*, vol. abs/2008.13456, 2020. [Online]. Available: <https://arxiv.org/abs/2008.13456>
- [27] H. Ng, S. Haridi, and P. Carbone, “Omni-paxos: Breaking the barriers of partial connectivity,” in *Proceedings of the Eighteenth European Conference on Computer Systems*, ser. EuroSys '23. New York, NY, USA: Association for Computing Machinery, 2023. doi: 10.1145/3552326.3587441. ISBN 9781450394871 p. 314–330. [Online]. Available: <https://doi.org/10.1145/3552326.3587441>
- [28] X. Zhou, B. Chen, Y. Gui, and L. Cheng, “Conformal prediction: A data perspective,” *ACM Comput. Surv.*, vol. 58, no. 2, Sep. 2025. doi: 10.1145/3736575. [Online]. Available: <https://doi.org/10.1145/3736575>

- [29] A. N. Angelopoulos, R. F. Barber, and S. Bates, “Theoretical foundations of conformal prediction,” 2025. [Online]. Available: <https://arxiv.org/abs/2411.11824>
- [30] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*, 2nd ed. Springer Cham, 2022. ISBN 978-3-031-06649-8. [Online]. Available: <https://doi.org/10.1007/978-3-031-06649-8>
- [31] I. Gibbs and E. Candes, “Adaptive conformal inference under distribution shift,” in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 1660–1672. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2021/file/0d441de75945e5acbc865406fc9a2559-Paper.pdf
- [32] M. Zaffran, O. Feron, Y. Goude, J. Josse, and A. Dieuleveut, “Adaptive conformal predictions for time series,” in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 17–23 Jul 2022, pp. 25 834–25 866. [Online]. Available: <https://proceedings.mlr.press/v162/zaffran22a.html>
- [33] S. Feldman, L. Ringel, S. Bates, and Y. Romano, “Achieving risk control in online learning settings,” 2023. [Online]. Available: <https://arxiv.org/abs/2205.09095>
- [34] L. Lamport, “Generalized consensus and paxos,” Microsoft Research, Tech. Rep. MSR-TR-2005-33, March 2005. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/generalized-consensus-and-paxos/>
- [35] B. Arun, S. Peluso, R. Palmieri, G. Losa, and B. Ravindran, “Speeding up consensus by chasing fast decisions,” in *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2017. doi: 10.1109/DSN.2017.35 pp. 49–60. [Online]. Available: <http://doi.org/10.1109/DSN.2017.35>
- [36] V. Enes, C. Baquero, T. F. Rezende, A. Gotsman, M. Perrin, and P. Sutra, “State-machine replication for planet-scale systems,” in *Proceedings of the Fifteenth European Conference on Computer Systems*, ser. EuroSys ’20. New York, NY, USA: Association for Computing Machinery,

2020. doi: 10.1145/3342195.3387543. ISBN 9781450368827. [Online]. Available: <https://doi.org/10.1145/3342195.3387543>
- [37] S. Tollman, S. J. Park, and J. Ousterhout, “EPaxos revisited,” in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, Apr. 2021. ISBN 978-1-939133-21-2 pp. 613–632. [Online]. Available: <https://www.usenix.org/conference/nsdi21/presentation/tollman>
- [38] A. Ganesan, R. Alagappan, A. Rebello, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “Exploiting nil-external interfaces for fast replicated storage,” *ACM Trans. Storage*, vol. 18, no. 3, Sep. 2022. doi: 10.1145/3542821. [Online]. Available: <https://doi.org/10.1145/3542821>
- [39] M. Burke, A. Cheng, and W. Lloyd, “Gryff: Unifying consensus and shared registers,” in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Santa Clara, CA: USENIX Association, Feb. 2020. ISBN 978-1-939133-13-7 pp. 591–617. [Online]. Available: <https://www.usenix.org/conference/nsdi20/presentation/burke>
- [40] H. Howard, D. Malkhi, and A. Spiegelman, “Flexible paxos: Quorum intersection revisited,” 2016. [Online]. Available: <https://arxiv.org/abs/1608.06696>
- [41] R. Frank, O. Lomeli, N. Giridharan, S. Ponnappalli, M. K. Aguilera, and N. Crooks, “Real life is uncertain. consensus should be too!” in *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems*, ser. HotOS ’25. New York, NY, USA: Association for Computing Machinery, 2025. doi: 10.1145/3713082.3730374. ISBN 9798400714757 p. 80–87. [Online]. Available: <https://doi.org/10.1145/3713082.3730374>
- [42] K. Shiozaki and J. Nakamura, “Dynatune: Dynamic tuning of raft election parameters using network measurement,” in *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, June 2025. doi: 10.1109/IPDPSW66978.2025.00091. ISSN 2995-066X pp. 568–577. [Online]. Available: <https://doi.org/10.1109/IPDPSW66978.2025.00091>

- [43] G. Hu, Y. Chen, A. Arpaci-Dusseau, and R. Arpaci-Dusseau, “Crossword: Adaptive consensus for dynamic data-heavy workloads,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.07157>
- [44] Y. Li, X. Yu, N. Koudas, S. Lin, C. Sun, and C. Chen, “dbet: Execution time distribution-based plan selection,” *Proc. ACM Manag. Data*, vol. 1, no. 1, May 2023. doi: 10.1145/3588711. [Online]. Available: <https://doi.org/10.1145/3588711>
- [45] H. Liu, S. Giridhara, and I. Sabek, “Conformal prediction for verifiable learned query optimization,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.02284>
- [46] S. Horchidan, F. Zeiher, H. Boström, and P. Carbone, “Conann: Conformal approximate nearest neighbor search,” *Proc. VLDB Endow.*, vol. 19, no. 1, p. 29–42, Jan. 2026. doi: 10.14778/3772181.3772184. [Online]. Available: <https://doi.org/10.14778/3772181.3772184>
- [47] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’16. New York, NY, USA: Association for Computing Machinery, 2016. doi: 10.1145/2939672.2939785. ISBN 9781450342322 p. 785–794. [Online]. Available: <https://doi.org/10.1145/2939672.2939785>
- [48] A. Angelopoulos, E. Candes, and R. J. Tibshirani, “Conformal pid control for time series prediction,” in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023. doi: 10.52202/075280-1000 pp. 23 047–23 074. [Online]. Available: <https://doi.org/10.52202/075280-1000>
- [49] H. Ng, K. Wu, and P. Carbone, “Unicache: Efficient log replication through learning workload patterns,” in *Proceedings of the 26th International Conference on Extending Database Technology (EDBT)*. OpenProceedings.org, 2023. doi: 10.48786/edbt.2023.39 pp. 471–477. [Online]. Available: <https://doi.org/10.48786/edbt.2023.39>

Appendix A

Proof of Slow-Path Resolution Safety

Let n denote the total number of nodes in the cluster, $Q_M = \lceil \frac{n+1}{2} \rceil$ represent the standard majority quorum, and $Q_F = \lceil \frac{3n}{4} \rceil$ represent the fast quorum. Assume the leader has collected Q_F total responses for a given slot. Consider the worst-case scenario where these responses are partitioned entirely between a single value A marked as `LEADERACCEPT` and a single value B marked as `FASTACCEPT`. Let v_A and v_B denote the observed vote counts for A and B respectively, such that $v_A + v_B = Q_F$. The number of uncast remaining votes is bounded by $R = n - Q_F$.

For both values to remain viable candidates for quorum satisfaction, the remaining votes must be sufficient to independently drive both values to their respective thresholds. This yields the following system of inequalities:

$$v_A + R \geq Q_M \tag{A.1}$$

$$v_B + R \geq Q_F \tag{A.2}$$

Summing inequalities (A.1) and (A.2) and substituting $v_A + v_B = Q_F$ results in:

$$Q_F + 2R \geq Q_M + Q_F \implies 2R \geq Q_M \tag{A.3}$$

Substituting $R = n - Q_F$ into the expression yields the following:

$$2(n - Q_F) \geq Q_M \implies 2n \geq Q_M + 2Q_F \tag{A.4}$$

By applying the definitions of the quorum thresholds ($Q_M \geq \frac{n+1}{2}$ and $Q_F \geq$

$\frac{3n}{4}$), we obtain the following:

$$2n \geq \frac{n+1}{2} + 2\left(\frac{3n}{4}\right) \implies 2n \geq 2n + \frac{1}{2} \implies 0 \geq \frac{1}{2} \quad (\text{A.5})$$

This yields a clear contradiction. Thus, given Q_F total responses, it is mathematically impossible for both value A to retain a path to a standard majority quorum and value B to retain a path to a fast quorum. Consequently, the leader can always unambiguously determine that at least one of the protocol paths has failed to achieve its quorum, allowing for a safe slow-path resolution.

Appendix B

Declaration of AI Use

The author declares that generative artificial intelligence was used exclusively for language polishing and stylistic enhancement throughout the drafting process. This support was strictly limited to refining grammar, phrasing, and overall readability. All fundamental research, technical design, data analysis, and conclusions remain entirely the author's own original work, and the author takes full responsibility for the content and integrity of this thesis.

